

Minimizing Mismatch Risk: A Prototype-Based Routing Framework for Zero-shot LLM-generated Text Detection

Ke Sun¹ Guangsheng Bao¹ Han Cui¹ Yue Zhang*¹

Abstract

Zero-shot methods detect LLM-generated text by computing statistical signatures using a surrogate model. Existing approaches typically employ a fixed surrogate for all inputs regardless of the unknown source. We systematically examine this design and find that detection performance varies substantially depending on surrogate-source alignment. We observe that while no single surrogate achieves optimal performance universally, a well-matched surrogate typically exists within a diverse pool for any given input. This finding transforms robust detection into a routing problem: selecting the most appropriate surrogate for each input. We propose DetectRouter, a prototype-based framework that learns text-detector affinity through two-stage training. The first stage constructs discriminative prototypes from white-box models; the second generalizes to black-box sources by aligning geometric distances with observed detection scores. Experiments on EvoBench and MAGE benchmarks demonstrate consistent improvements across multiple detection criteria and model families. Our Codes are available in [DetectRouter code](#).

1. Introduction

The rapid proliferation of Large Language Models has revolutionized natural language generation, enabling applications ranging from creative content to code synthesis (Vaswani et al., 2017; Ho et al., 2020; Sun et al., 2025a; Ma et al., 2024; 2026). However, this capability poses significant risks to information integrity, including the mass production of disinformation, academic dishonesty, and the contamination of web corpora used to train future models (Milano et al., 2023; Demszky et al., 2023; Doshi

¹School of Engineering, Westlake University, China. Correspondence to: Yue Zhang <zhangyue@westlake.edu.cn>.

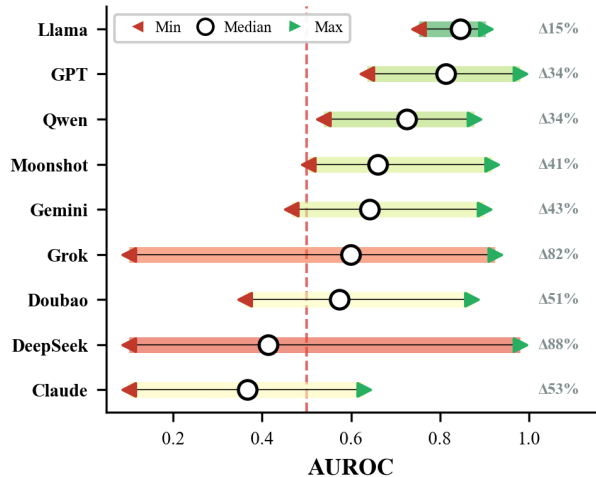


Figure 1. Detection variance of Fast-DetectGPT across black-box LLM families on the MIRAGE benchmark. Each bar spans from the minimum to maximum AUROC achieved by different surrogate models, with circles indicating the median. The percentage on the right denotes the performance gap between the best and worst surrogate for each source.

& Hauser, 2024). Consequently, distinguishing between human-written and machine-generated text has become an imperative safety challenge (Yang et al., 2024; Wu et al., 2025; Zhou et al., 2024; 2025; Sun et al., 2024).

Among existing detection approaches, zero-shot methods have gained prominence for their training-free nature. These methods employ a surrogate LLM to compute token-level statistics of the candidate text, such as conditional probability or likelihood curvature, and aggregate these statistics into a detection score that discriminates machine-generated content from human writing (Mitchell et al., 2023; Bao et al., 2024; Sun et al., 2026). While such methods achieve strong performance in the white-box setting where the source model is accessible, their effectiveness degrades substantially in real-world black-box scenarios where the generating model is unknown (Fu et al., 2025a; Bao et al., 2025). This gap presents an urgent challenge that demands systematic investigation.

A key factor underlying this limitation is that all existing

zero-shot methods employ a fixed surrogate model for detection across diverse text sources, justified by the distribution matching hypothesis that probability distributions across different LLMs are similar (Mitchell et al., 2023; Bao et al., 2024). Our preliminary experiments challenge this assumption. As shown in Figure 1, evaluating Fast-DetectGPT with nine open-source surrogate models on the MIRAGE benchmark (Fu et al., 2025a) reveals dramatic performance variance: for challenging sources such as DeepSeek and Grok, the gap between the best and worst surrogate exceeds 80%, spanning from near-random to strong detection. We further quantify this phenomenon in Section 3 and derive the Mismatch Risk Bound, which establishes that the performance gap is upper-bounded by the square root of the KL divergence between source and surrogate distributions. This bound transforms detection into a routing problem: given an input text, we must identify the surrogate that minimizes distributional divergence to the unknown source.

Building on this insight, we propose DetectRouter, a prototype-based routing framework for zero-shot LLM detection. Given a text of unknown origin, DetectRouter first encodes it into a learned metric space, then identifies the nearest prototype among a set of detector-specific representations, and finally routes the text to the corresponding optimal detector for scoring. Since true source distributions are inaccessible for black-box models, we approximate distributional affinity geometrically: we learn an embedding space where distances to detector prototypes reflect underlying distributional proximity. DetectRouter employs a two-stage training strategy. Stage 1 constructs discriminative prototypes using white-box models with known source labels through contrastive learning. Stage 2 generalizes to black-box sources by treating detection scores from the surrogate pool as supervision and minimizing the KL divergence between predicted affinity and observed score distributions. At inference time, the router routes each input to its most compatible detector via nearest-prototype matching.

Extensive experiments on EvoBench and MAGE benchmarks demonstrate that DetectRouter consistently improves all six detection criteria, with relative gains ranging from 5.4% to 139.4% and averaging 36.1% over fixed-surrogate baselines. DetectRouter achieves state-of-the-art performance with 90.85% average AUROC on EvoBench and 77.92% on MAGE, outperforming the fixed-surrogate best baseline by 9.84% and 8.23% points respectively.

Our contributions are summarized as follows:

- We conduct the first systematic analysis of surrogate selection in zero-shot LLM detection, revealing that surrogate-source affinity is the decisive factor for black-box performance.
- We propose DetectRouter, a prototype-based routing

framework that learns to match input texts to optimal detectors through two-stage training combining discriminative prototype construction with distributional alignment.

- Extensive experiments on two benchmarks demonstrate that DetectRouter achieves state-of-the-art performance and serves as a universal enhancement layer for all zero-shot detection methods.

Conflict of Interest Disclosure. The authors declare no financial conflicts of interest.

2. Related Work

Text generation detection. Existing methods fall into two categories. Training-based methods fine-tune classifiers with ensemble, contrastive, or adversarial objectives (Abburi et al., 2023; Hu et al., 2023; Guo et al., 2024; Chen et al., 2025; Su et al., 2023b), but often degrade on unseen models or prompts (Yang et al., 2024; Wu et al., 2025; Sun et al., 2025b; Wu et al., 2026; Liu et al., 2026; Zhu et al., 2026). Statistics-based methods instead treat a surrogate LLM as a scoring function and design detection rules from token probabilities, yielding zero-shot detectors without task-specific data. GLTR uses token-level statistics such as log-likelihood, entropy, and rank histograms (Gehrmann et al., 2019); DetectLLM exploits log-rank scores (Su et al., 2023a); DetectGPT measures log-likelihood curvature under perturbations (Mitchell et al., 2023); and Fast-DetectGPT replaces sampling with an efficient analytic approximation (Bao et al., 2024). Recent extensions include kernel-based tests (Yu et al., 2024; Song et al., 2025) and memorization-based detectors (Guo et al., 2024).

Routing and origin tracing for LLMs. Model routing learns to assign inputs to appropriate models from a pool with varying accuracy-cost trade-offs, enabling efficient inference under budget constraints (Hu et al., 2024; Huang et al., 2025; Jitkrittum et al., 2025; Chuang et al., 2024). However, existing approaches primarily treat LLMs as task solvers for QA or reasoning, rather than as detectors for generated text. Origin-tracing methods such as Li et al. (2023) and Fu et al. (2025b) build classifiers to identify which LLM produced a given text in black-box scenarios, but their goal is provenance attribution rather than exploiting detector complementarity for improved detection. Sun et al. (2025b) show that different LLMs exhibit highly separable idiosyncratic patterns, enabling source recovery with high accuracy.

3. Analysis: The Cost of Misalignment

Before presenting our method, we investigate the validity of the Distribution Matching Hypothesis that underpins

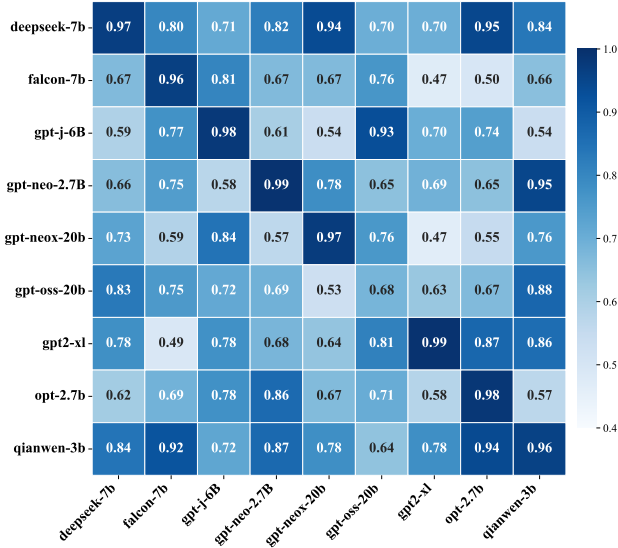


Figure 2. Affinity matrix of detection performance using Fast-DetectGPT. Nine open-source models serve as both generators (rows) and surrogates (columns). Diagonal dominance confirms optimal performance requires source-surrogate alignment, while off-diagonal patterns reveal structured affinity governed by architectural similarity.

current zero-shot detection approaches. Standard methods employ a fixed surrogate model to estimate statistical artifacts, assuming its probability landscape sufficiently approximates diverse source distributions (Mitchell et al., 2023; Bao et al., 2024). While appealing in simplicity, this assumption overlooks *model idiosyncrasies*—distinct statistical fingerprints arising from varying architectures and pre-training corpora (Sun et al., 2025b; Yang et al., 2024).

3.1. Experimental Setup

We design cross-evaluation experiments in both white-box and black-box settings. For the white-box setting, we follow the protocol of Fast-DetectGPT (Bao et al., 2024) and use XSum (Narayan et al., 2018) as the seed corpus. Nine open-source LLMs ranging from 2.7B to 20B parameters each generate 500 text continuations via nucleus sampling. We then evaluate every model as a surrogate detector on all nine corpora, yielding a 9×9 cross-evaluation matrix. For the black-box setting, we utilize the MIRAGE benchmark (Fu et al., 2025a) with proprietary models including GPT-4, Claude, Gemini, and DeepSeek. Complete experimental details are provided in Appendix E.

3.2. Empirical Investigation

White-box Setting. Figure 2 visualizes the pairwise AUC scores for Fast-DetectGPT. The heatmap reveals two critical patterns. First, we observe pronounced diagonal dominance, confirming that detectors achieve optimal performance only

Table 1. Quantifying source-surrogate misalignment across five detection criteria. Matched: source = surrogate; Cross: source $\neq$ surrogate. For black-box setting, Max/Min denote the best/worst surrogate averaged across all generator families.

Criterion	White-box / XSum		Black-box / MIRAGE		
	Matched	Cross	Mean	Max	Min
Likelihood	0.81	0.42	0.49	0.76	0.35
Rank	0.76	0.60	0.52	0.64	0.43
LogRank	0.85	0.46	0.48	0.73	0.35
Entropy	0.55	0.71	0.59	0.74	0.41
Fast-DetectGPT	0.94	0.71	0.65	0.93	0.47
Average	0.78	0.58	0.55	0.76	0.40

when source equals surrogate. Second, the off-diagonal degradation is structured: surrogates sharing architectural lineage, such as GPT-Neo and GPT-J families, maintain high mutual affinity, whereas structurally distinct pairs exhibit sharp performance collapse. Notably, even among models of comparable scale, architectural differences alone can cause AUC drops exceeding 30 percentage points. We provide affinity matrices for all five detection criteria in Appendix C.1.

Black-box Setting. As illustrated in Figure 1, the performance variance across different surrogate choices is extreme. For challenging sources like Claude and DeepSeek, AUC spans from near-random levels around 0.2 to near-perfect detection exceeding 0.9, depending solely on surrogate selection. While the median performance hovers near random baseline, the maximum consistently remains high—confirming that detection capability exists within the open-source community but is fragmented across models. This fragmentation implies that no single surrogate universally dominates, necessitating adaptive selection based on input characteristics. Extended visualizations including per-source distributions are provided in Appendix C.2.

3.3. Quantitative Analysis

Table 1 quantifies the cost of source-surrogate misalignment across five detection criteria. In the white-box setting, average AUC drops from 0.78 in the matched condition to 0.58 in the cross-model condition, representing a 26% relative decline. For Fast-DetectGPT, performance degrades from 0.94 to 0.71—a gap that transforms a near-perfect detector into a mediocre one.

In the black-box setting, variance across surrogate choices is equally striking. While average performance is only 0.55, the best surrogate achieves 0.76 on average, confirming that optimal surrogates exist for each source. Thus, the spread is particularly pronounced: the best surrogate achieves 0.93 while the worst drops to 0.47. These findings establish that fixed surrogate selection incurs substantial penalties, yet op-

timal choices exist within a diverse model pool—motivating a routing mechanism for dynamic selection.

3.4. Theoretical Justification

The empirical results highlight a consistent pattern: detection reliability is inversely correlated with the distributional distance between surrogate and source. To formalize this, we establish the following notation. Let P_{src} denote the probability distribution of the source model that generated the text, and let P_{sur} denote the distribution of the surrogate model used for detection. Given a bounded scoring function $T(x) : \mathcal{X} \rightarrow \mathbb{R}$ with $|T(x)| \leq B$, the optimal detection signal is $\mu^* = \mathbb{E}_{P_{src}}[T(x)]$, while the proxy signal obtained using the surrogate is $\mu_{proxy} = \mathbb{E}_{P_{sur}}[T(x)]$.

Proposition 3.1 (Mismatch Risk Bound). *For any bounded zero-shot detection statistic T with $|T(x)| \leq B$:*

$$|\mu^* - \mu_{proxy}| \leq B \cdot \sqrt{2D_{KL}(P_{src}||P_{sur})} \quad (1)$$

where D_{KL} denotes the Kullback-Leibler divergence.

The proof proceeds via Total Variation distance and Pinsker’s inequality; we provide the complete derivation in Appendix A.

Scope and design implications. Proposition 3.1 bounds the *magnitude*, not direction, of score distortion for any single detector; end-to-end AUROC guarantees are deferred to empirical validation. The bound nonetheless motivates two design choices: (i) since different detectors incur different mismatch risks across inputs, no single surrogate is universally optimal—hence *per-input* selection; (ii) we operationalize affinity matching by aligning the predicted distribution with the observed score distribution via KL divergence, which outperforms L_1/L_2 /cosine alternatives by 3.4–4.3 points (Appendix C.4). Empirically, prototype distances correlate with both AUROC ($\rho = -0.76$) and token-level KL ($r = 0.72$, Appendix C.3).

4. Methodology

4.1. Problem Formulation

Our analysis in Section 3 suggests that the optimal strategy is to dynamically select a surrogate M_k minimizing $D_{KL}(P_{src}||P_{M_k})$. This reframes detection as a routing problem as shown in Figure 3: given text x from unknown source M_{src} , identify the optimal detector from pool $\mathcal{D} = \{D_1, \dots, D_N\}$, where each D_k pairs a surrogate M_k with a detection criterion $F(\cdot; M_k)$ such as Likelihood or Fast-DetectGPT:

$$k^* = \arg \min_{k \in \{1, \dots, N\}} D_{KL}(P_{src}||P_{M_k}) \quad (2)$$

Since P_{src} is inaccessible for black-box models, we approximate distributional affinity geometrically. We learn an encoder $E_\theta : \mathcal{X} \rightarrow \mathbb{R}^d$ and represent each detector D_k with learnable prototypes $\mathcal{P}_k = \{\mathbf{p}_{k,1}, \dots, \mathbf{p}_{k,K}\}$ capturing the signatures of model family k . The routing decision becomes a nearest-prototype query:

$$k^* = \arg \min_k \min_{\mathbf{p} \in \mathcal{P}_k} \|E_\theta(x) - \mathbf{p}\|_2 \quad (3)$$

We employ two-stage training: Stage 1 builds discriminative prototypes from white-box models, while Stage 2 aligns black-box sources via distributional alignment.

4.2. Stage 1: Discriminative Prototype Construction

The first stage establishes a well-structured metric space using N white-box models from the model pool $\mathcal{M}$ where ground-truth source labels are available. A key design consideration is ensuring that prototypes capture intrinsic stylistic fingerprints rather than task-specific artifacts. We construct a diverse anchor dataset $\mathcal{T}_1 = \{(x_j, y_j)\}_{j=1}^{|\mathcal{T}_1|}$ through multi-task generations: each model M_i is tasked with (1). direct generation from seed prompts, (2). text polishing of human drafts, and (3). rewriting of existing passages. This diversity forces the encoder to identify model-specific patterns that persist across varying conditions, yielding prototypes that generalize beyond any single task.

Distance-Based Classification. We quantify the affinity between an input x , embedded as $\mathbf{z} = E_\theta(x)$, and a detector class i via the distance to its nearest prototype:

$$d_i(\mathbf{z}) = \min_{k \in \{1, \dots, K\}} \|\mathbf{z} - \mathbf{p}_{i,k}\|_2 \quad (4)$$

These geometric distances are transformed into classification probabilities using a temperature-scaled softmax, where a smaller distance implies higher affinity:

$$P(y = i | \mathbf{z}) = \frac{\exp(-d_i(\mathbf{z})/\tau)}{\sum_{j=1}^N \exp(-d_j(\mathbf{z})/\tau)} \quad (5)$$

Training Objectives. The encoder and prototypes are optimized jointly using a composite objective. We minimize the cross-entropy loss $\mathcal{L}_{CE}$ to cluster embeddings around their ground-truth prototypes:

$$\mathcal{L}_{CE} = -\frac{1}{|\mathcal{T}_1|} \sum_{(x,y) \in \mathcal{T}_1} \log P(y | E_\theta(x)) \quad (6)$$

To further structure the metric space, we introduce a margin-based separation loss $\mathcal{L}_{sep}$ that enforces inter-class separation by penalizing prototype pairs from different classes closer than margin m :

$$\mathcal{L}_{sep} = \frac{1}{Z} \sum_{i \neq j} \sum_{k,l} \max(0, m - \|\mathbf{p}_{i,k} - \mathbf{p}_{j,l}\|_2)^2 \quad (7)$$

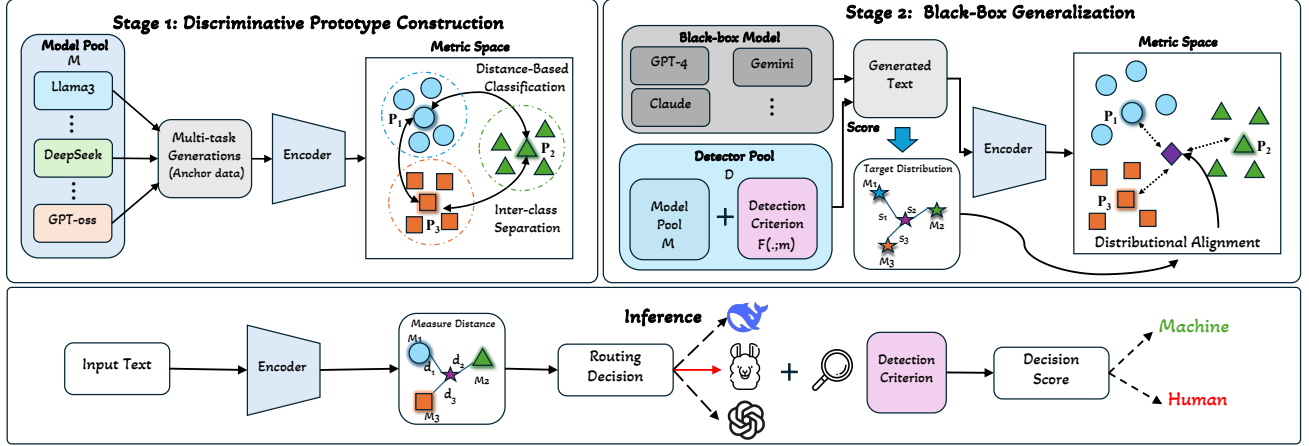


Figure 3. Overview of DetectRouter. Stage 1 constructs discriminative prototypes from white-box models via multi-task generations and distance-based classification. Stage 2 generalizes to black-box sources by aligning geometric distances with detection score distributions. At inference, the router selects the optimal detector based on nearest-prototype affinity.

A normalization term $\mathcal{L}_{\text{norm}} = \sum_{i,k} \|\mathbf{p}_{i,k}\|_2^2$ constrains prototype magnitude for training stability. The complete Stage 1 objective is:

$$\mathcal{L}_1 = \mathcal{L}_{\text{CE}} + \lambda_{\text{sep}} \mathcal{L}_{\text{sep}} + \lambda_{\text{norm}} \mathcal{L}_{\text{norm}} \quad (8)$$

Upon convergence, this yields a discriminative embedding space characterized by high intra-class compactness and clear inter-class separation.

4.3. Stage 2: Black-Box Generalization via Distributinal Alignment

While Stage 1 builds the coordinate system, it relies on explicit labels. Real-world scenarios involve unknown black-box models such as GPT-4 where labels are inaccessible. To bridge this gap, we leverage the vector of detection scores produced by the detector pool $\mathcal{D}$ as an empirical proxy for distributinal alignment.

Target Distribution Construction. For a black-box text x , we observe a score vector $\mathbf{s} = [s_1, \dots, s_N]$ from the detector pool. These scores reflect the text’s statistical proximity to each known model family. Instead of regressing absolute scores, which can be unstable due to scale differences, we normalize them into a target distribution Q via temperature-scaled softmax:

$$Q_i = \frac{\exp(s_i/T)}{\sum_{j=1}^N \exp(s_j/T)} \quad (9)$$

Simultaneously, the router outputs a predicted affinity distribution P derived from the geometric distances in the prototype space as defined in Eq. 5.

KL-Divergence Optimization. We formulate the learning objective as a distribution matching problem, minimizing

the Kullback-Leibler divergence between the target distribution Q and the predicted geometric distribution P :

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}(Q \| P) = \sum_{i=1}^N Q_i \log \frac{Q_i}{P_i} \quad (10)$$

This objective explicitly aligns the router’s geometric logic with the theoretical goal of minimizing mismatch risk: the router learns to position the black-box embedding such that its distances to prototypes mirror the actual detection affinities.

Prototype Anchoring. Unsupervised adaptation risks catastrophic forgetting, where prototypes might drift and destroy the discriminative structure learned in Stage 1. To prevent this, we employ prototype anchoring by freezing a copy of the converged Stage 1 prototypes $\{\mathbf{p}^{\text{anchor}}\}$ and penalizing deviations:

$$\mathcal{L}_{\text{anchor}} = \sum_{i,k} \|\mathbf{p}_{i,k} - \mathbf{p}_{i,k}^{\text{anchor}}\|_2^2 \quad (11)$$

We additionally retain the cross-entropy term $\mathcal{L}_{\text{CE}}$ on the human class, whose source label remains valid in Stage 2, to preserve the discriminative geometry around the human prototype. The complete Stage 2 objective combines distributinal alignment with regularization:

$$\mathcal{L}_2 = \mathcal{L}_{\text{CE}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}} + \lambda_{\text{anc}} \mathcal{L}_{\text{anchor}} + \lambda_{\text{sep}} \mathcal{L}_{\text{sep}} + \lambda_{\text{norm}} \mathcal{L}_{\text{norm}} \quad (12)$$

This ensures the router generalizes to unseen sources while retaining the robust topology of known models.

4.4. Inference

During inference, given an input text x , the encoder projects it into the metric space to calculate the distances

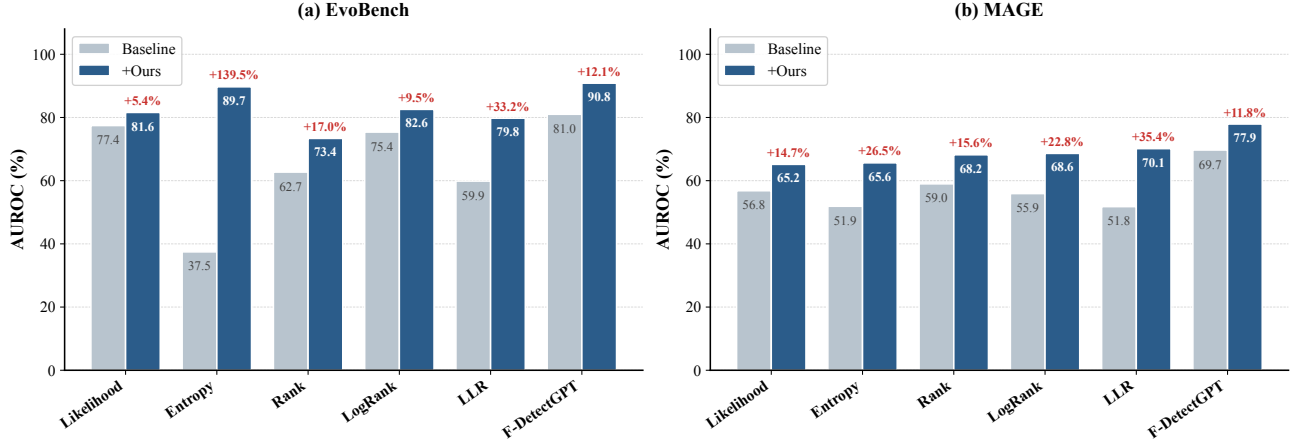


Figure 4. Performance improvement from applying DetectRouter to each zero-shot detection method. Bars show average AUROC on EvoBench and MAGE; numbers indicate relative improvement.

$\{d_1, \dots, d_N\}$ against all detector prototypes. The routing decision identifies the detector with the highest affinity by solving $k^* = \arg \min_k d_k$. The input is then routed to the selected detector D_{k^*} , which computes the final detection score to classify the text as human-written or machine-generated.

5. Experiments

5.1. Experimental Setup

Training Data. We train DetectRouter on the MIRAGE benchmark (Fu et al., 2025a), a comprehensive multi-task dataset spanning five domains—Academic, News, Comment, Email, and Website—and three generation tasks: *Generate*, *Polish*, and *Rewrite*. For Stage 1, we synthesize 30K training samples by prompting locally cached surrogate models to generate text aligned with the MIRAGE schema. For Stage 2, we leverage MIRAGE directly with detection scores annotated across our full detector pool, providing supervision for learning text-detector affinity patterns.

Surrogate Models and Detection Methods. Our surrogate pool comprises ten open-source LLMs spanning 1.5B to 20B parameters: GPT-J-6B (Wang & Komatsuzaki, 2021), GPT-NeoX-20B (Biderman et al., 2023), GPT-Neo-2.7B (Black et al., 2021), GPT2-XL (Radford et al., 2019), GPT-oss-20B (OpenAI, 2025), LLaMA3-8B (Dubey et al., 2024), Falcon-7B (Almazrouei et al., 2023), DeepSeek-7B (DeepSeek-AI, 2023), OPT-2.7B (Zhang et al., 2022), and Qwen-3B (Bai et al., 2023). This selection ensures diversity across architectures and training corpora. We evaluate six zero-shot detection criteria—Likelihood, Entropy, Rank, LogRank, LLR, and Fast-DetectGPT (Bao et al., 2024)—comparing each fixed-surrogate baseline against our routed variant. We first benchmark all detection methods across

all ten surrogate models and identify LLaMA3-8B as the consistently best-performing surrogate; we therefore fix all baseline methods to use LLaMA3-8B, ensuring the most competitive comparison rather than an artificially weakened baseline.

Router Configuration. The router employs LLM2Vec-LLaMA3-8B (BehnamGhader et al., 2024) as the encoder for main experiments, with lightweight alternatives explored in ablation studies. Each class is represented by 10 learnable prototypes with cosine similarity and temperature $\tau=10$. Both training stages run for 8 epochs with batch size 32; Stage 1 uses learning rate 2×10^{-5} while Stage 2 reduces to 10^{-5} and balances cross-entropy, distance regression, and prototype anchoring losses.

Baselines. We compare against three groups of methods. (1) **Classical zero-shot detectors** use token-level statistics computed from a fixed surrogate, including Likelihood (Sollaiman et al., 2019), Entropy and Rank (Gehrmann et al., 2019), LogRank (Mitchell et al., 2023), relative scoring via LLR (Su et al., 2023a), and perturbation-based Fast-DetectGPT (Bao et al., 2024). (2) **Recently published detectors** span diverse paradigms: Lastde (Xu et al., 2024b) mines local temporal dynamics in token probability sequences; FourierGPT (Xu et al., 2024a) analyzes spectral signatures of likelihood; Diveye (Basani & Chen, 2025) aggregates diversity statistics; UCE (Hou et al., 2025) provides a theoretical detection framework; MMD-MP (Zhang et al., 2024) and Deep Kernel Relative Test (Song et al., 2025) are kernel two-sample tests with multi-population and relative-test optimization respectively; and Easy-to-Hard (Wu et al., 2026) enhances detectors via supervision from longer-text auxiliaries. (3) **Supervised approaches** include classifiers trained on our Stage 2 data using each surrogate’s representation, and the official DetectAnyLLM (Fu et al., 2025a)

Table 2. Detection performance on EvoBench and MAGE. Fixed-surrogate baselines use LLaMA3-8B. “Supervised” is our supervised classifier trained on Stage 2 data with LLaMA3-8B representations. “DetectAnyLLM” is the official checkpoint released by Fu et al. (2025a).

Method	EvoBench								MAGE
	LLaMA3	LLaMA2	Claude	GPT-4o	Gemini	GPT-4	Qwen	Avg	Avg
Likelihood	88.84	61.68	79.70	77.80	78.89	77.80	77.40	77.44	56.81
Entropy	28.85	57.58	33.61	35.43	38.34	38.10	30.34	37.46	51.90
Rank	68.49	58.38	64.50	64.64	64.46	63.43	55.06	62.71	58.99
LogRank	86.47	59.99	77.37	76.80	75.99	74.87	76.10	75.37	55.91
LLR	66.46	49.61	61.73	60.57	59.12	58.43	63.08	59.86	51.78
Fast-DetectGPT	94.17	80.27	81.57	74.89	80.48	75.58	80.10	81.01	69.69
Lastde	90.80	80.54	76.75	71.34	76.50	71.51	69.20	76.66	70.71
FourierGPT	63.99	57.23	58.20	55.52	56.69	55.16	64.39	58.74	60.34
Diveye	84.65	73.46	74.21	73.32	74.06	72.28	68.85	74.40	69.60
UCE	75.62	61.35	68.72	62.04	61.28	59.38	63.63	64.57	54.17
MMD-MP	84.21	73.85	79.36	78.84	76.92	79.21	74.66	78.15	72.45
Deep Kernel	89.42	80.73	85.18	83.27	83.05	82.69	81.71	83.72	74.18
Easy-to-Hard+FDGPT	95.03	81.94	84.32	79.16	83.46	80.21	83.11	83.89	74.62
Supervised	74.96	66.88	73.38	77.66	62.07	76.16	66.49	71.09	74.69
DetectAnyLLM	82.34	57.92	82.94	87.09	80.51	86.48	66.64	77.70	70.11
DetectRouter	95.66	80.82	94.22	94.29	93.79	94.16	83.01	90.85	77.92

checkpoint, which is trained on the MIRAGE training split via Direct Discrepancy Learning. For zero-shot baselines that rely on a surrogate model, we use LLaMA3-8B—the strongest single surrogate identified above—to ensure the most competitive comparison.

Evaluation Benchmarks. We evaluate on two complementary in-the-wild benchmarks. MAGE (Li et al., 2024) provides broad coverage with 30,265 test pairs across eight model families—OpenAI, OPT, FLAN, LLaMA, BigScience, EleutherAI, GLM, and human paraphrase—representing established generators and diverse domains. EvoBench (Yu et al., 2025) specifically targets the evolving nature of LLMs, comprising 32,993 pairs across seven families and 29 evolving versions, capturing both publisher updates and developer modifications such as fine-tuning and pruning. Following prior work (Mitchell et al., 2023; Bao et al., 2024), we report AUROC as the primary metric. Complete details on data construction, hyperparameters, and benchmark statistics are provided in Appendix D.

5.2. Main Results

A central contribution of DetectRouter is its ability to serve as a universal enhancement layer for any zero-shot detection criterion. We first demonstrate the universal enhancement property before presenting state-of-the-art comparisons.

Routing as a Universal Enhancement. Figure 4 visualizes the performance gains achieved by applying our

Table 3. Ablation on training stages and loss components. Average AUROC is reported in percentage. Δ denotes relative drop from the full method.

Setting	EvoBench	MAGE	Δ
Fixed Surrogate	81.01	69.69	-10.7%
Direct Classification	83.08	75.34	-5.9%
Stage 1 Only	72.87	75.67	-11.3%
Stage 2 Only	87.39	74.49	-4.1%
Full w/o $\mathcal{L}_{KL}$	87.07	75.56	-3.6%
Full w/o $\mathcal{L}_{sep}$	88.53	74.41	-3.5%
Full w/o $\mathcal{L}_{anchor}$	89.56	74.90	-2.6%
Full	90.85	77.92	—

routing mechanism to each of the six detection methods. On EvoBench, routing provides relative improvements ranging from 5.4% for Likelihood to 139.4% for Entropy. On MAGE, improvements range from 11.8% for Fast-DetectGPT to 35.4% for LLR. Entropy-based detection benefits most dramatically: the fixed-surrogate baseline achieves only 37.46% on EvoBench, falling below random chance for several model families, while routing elevates performance to 89.70%. This demonstrates that our framework is not tied to any specific detection criterion but serves as a general-purpose enhancement layer that can be seamlessly integrated with existing methods. Per-family breakdown for all method combinations is provided in Appendix B.2.

Table 4. Ablation on router encoder architectures. Average AUROC is reported in percentage alongside model parameters and inference latency per sample.

Encoder	EvoBench	MAGE	Params	Latency
Fixed Surrogate	81.01	69.69	—	—
BERT-base	86.41	71.07	109M	4.87ms
RoBERTa-base	88.34	74.10	125M	4.87ms
RoBERTa-large	88.18	75.59	355M	12.70ms
LLM2Vec-LLaMA3	90.85	77.92	8B	166.2ms

Comparison with Baselines. Table 2 presents AUROC scores on EvoBench with per-family breakdown and MAGE average. DetectRouter achieves the best overall performance at 90.85% on EvoBench and 77.92% on MAGE, substantially outperforming all baselines. Among fixed-surrogate methods, Fast-DetectGPT is the strongest at 81.01% on EvoBench and 69.69% on MAGE; our routing mechanism improves upon this baseline by 9.84 and 8.23 percentage points respectively. The improvement is particularly pronounced on proprietary models where distributional gaps are largest: on Claude, performance increases from 81.57% to 94.22%; on GPT-4o, from 74.89% to 94.29%; and on GPT-4, from 75.58% to 94.16%. Among recent zero-shot methods, Easy-to-Hard+FDGPT achieves the highest average at 83.89% on EvoBench, yet DetectRouter outperforms it by 6.96 percentage points. FourierGPT, Diveye, and UCE all fall below our method by substantial margins, demonstrating that adaptive routing provides orthogonal benefits to detection criterion design. Full per-family results on MAGE are provided in Appendix B.1.

To isolate the contribution of our routing framework from the training data, we train supervised classifiers on the same Stage 2 data. Our supervised baseline using LLaMA3-8B representations achieves 71.09% on EvoBench and 74.69% on MAGE. DetectRouter outperforms this baseline by 19.76 percentage points on EvoBench and 3.23 points on MAGE, demonstrating that the gains stem from learned text-detector affinity patterns rather than simply utilizing more training data. Full results for all ten surrogate configurations are provided in Appendix B.3.

5.3. Ablation and Analysis

Training Stages and Loss Components. Table 3 presents ablation results. Both training stages prove essential: Stage 1 Only suffers −11.3% degradation as prototypes trained solely on white-box models fail to generalize, while Stage 2 Only shows a −4.1% gap from lacking well-initialized prototypes. Each loss component contributes meaningfully—replacing KL divergence with $\mathcal{L}_1$ regression causes −3.6% drop, and removing $\mathcal{L}_{sep}$ results in −3.5% degradation. Direct classification bypasses the prototype framework and

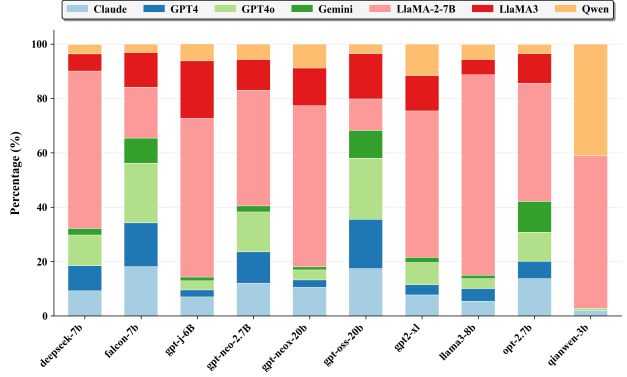


Figure 5. Distribution of selected surrogates for each source model family, confirming source-specific routing preferences.

exhibits −5.9% gap, confirming the advantage of metric-space routing.

Encoder Architecture. Table 4 compares different encoder architectures for the routing network. Among lightweight encoders, RoBERTa-base achieves the best balance, outperforming BERT-base by 1.93% on EvoBench and 3.03% on MAGE while maintaining identical inference latency at 4.87ms per sample. LLM2Vec-LLaMA3 achieves the best overall performance at 90.85% on EvoBench and 77.92% on MAGE, but requires 166.2ms per sample. This trade-off suggests that for latency-sensitive applications, RoBERTa-base offers an attractive operating point with strong performance at minimal computational overhead.

Routing Pattern Interpretation. To validate that our performance gains stem from adaptive alignment rather than simply ensembling, we visualize the distribution of selected surrogates for different source families in Figure 5. The stacked bar chart reveals that the router does not simply default to a single universal detector for all inputs. Instead, it exhibits distinct, source-specific preferences. For instance, when detecting text from DeepSeek-7B, the router frequently selects structurally similar surrogates, whereas for Falcon-7B, the preference shifts significantly toward a different subset of detectors. This confirms our hypothesis that affinity is a latent, continuous property: the router successfully identifies and exploits the unique statistical signatures of each source.

Data Efficiency. A key practical concern for black-box adaptation is the cost of collecting detection scores. We investigate the data efficiency of DetectRouter by varying the fraction of Stage 2 training data from 10% to 100%. As shown in Figure 6, the model exhibits remarkable stability. On EvoBench, the router achieves an AUC of 89.06% with only 10% of the data, within 1.8 percentage points of the performance achieved with the full dataset. This

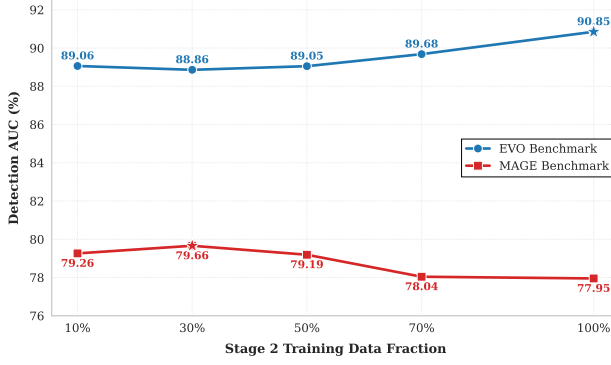


Figure 6. Detection AUC scores on EvoBench and MAGE plotted against the fraction of training data used in Stage 2.

Table 5. DetectRouter vs. max-selection and oracle baselines. Max-selection takes the highest oriented score across the surrogate pool; Oracle selects the best surrogate per sample with ground-truth source labels. AUROC in percentage.

Method	EvoBench	MAGE
Max-selection (per sample)	72.33	70.35
Fixed (LLaMA3-8B)	81.01	69.69
DetectRouter	90.85	77.92
Oracle (per-sample best)	94.76	80.47

indicates that the prototype space constructed in Stage 1 provides a strong regularization prior, making DetectRouter highly cost-effective for adapting to new models. Additional analysis on prototype space geometry is provided in Appendix B.4.

What DetectRouter Learns: Beyond Max-Selection.

A natural question is whether DetectRouter merely approximates max-selection over surrogates. Table 5 refutes this: max-selection ($\max_k s_k(x)$) drops to 72.33% on EvoBench—8.68 points below the fixed LLaMA3-8B baseline—because human text has no true source LLM, so the max across N surrogates inflates human scores via a multiple-testing effect, compressing the human-machine margin. DetectRouter avoids this by first inferring source affinity via nearest-prototype matching, then scoring with *only* the selected surrogate. This yields a stark routing asymmetry: human texts concentrate on a single surrogate (86.1% top-1, entropy 0.78 bits), while machine texts spread across the pool (49.1% top-1, entropy 2.12 bits), giving a 6.7% FPR for DetectRouter versus 74.2% for max-selection at the EER threshold. An oracle that selects the best surrogate per sample with ground-truth labels reaches 94.76%/80.47%; DetectRouter closes most of the fixed-to-oracle gap without any access to source identity at inference. Full analysis is in Appendix B.5.

6. Conclusion

This work reveals that zero-shot LLM detection is highly sensitive to the distributional alignment between the surrogate model and the unknown source. Through systematic experiments and theoretical analysis, we demonstrate that surrogate-source mismatch leads to substantial performance degradation, and derive a bound showing the gap is governed by KL divergence between distributions. Building on this finding, we propose DetectRouter, the first framework to leverage multiple surrogate models as complementary experts and dynamically route each input to its most compatible detector. Experiments confirm that this adaptive routing strategy yields consistent improvements across all detection criteria and achieves state-of-the-art performance on two benchmarks.

Acknowledgment

This work was financially supported by the NSFC key project 62336006.

Impact Statement

This work aims to advance the reliable detection of machine-generated text, which has broad implications for maintaining information integrity in the age of large language models. Effective detection methods can help combat the spread of AI-generated disinformation, preserve academic integrity, and prevent the contamination of training corpora with synthetic content. Our routing framework improves detection robustness across diverse and evolving LLMs, making it more practical for real-world deployment where the source model is unknown.

We acknowledge that detection technologies exist in an ongoing adversarial dynamic with generation methods. While our work strengthens the defensive capability, determined adversaries may develop countermeasures. We also note that imperfect detectors risk false accusations against human authors, which could have serious consequences in educational or professional contexts. We encourage practitioners to use detection tools as one signal among many rather than as definitive judgments, and to establish appropriate thresholds that balance false positive and false negative rates for their specific applications.

References

Abburi, H., Roy, K., Suesserman, M., Pudota, N., Veeramani, B., Bowen, E., and Bhattacharya, S. A simple yet efficient ensemble approach for AI-generated text detection. In Gehrmann, S., Wang, A., Sedoc, J., Clark, E., Dhole, K., Chandu, K. R., Santus, E., and Sedghamiz, H. (eds.), *Proceedings of the Third Workshop on Natural*

- Language Generation, Evaluation, and Metrics (GEM)*, pp. 413–421, Singapore, December 2023. Association for Computational Linguistics.
- Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., Debbah, M., Goffinet, É., Hesslow, D., Lounay, J., Malartic, Q., et al. The falcon series of open language models. *arXiv preprint arXiv:2311.16867*, 2023.
- Bai, J., Yang, A., Li, B., et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023. URL <https://arxiv.org/abs/2309.16609>.
- Bao, G., Zhao, Y., Teng, Z., Yang, L., and Zhang, Y. Fast-DetectGPT: Efficient zero-shot detection of machine-generated text via conditional probability curvature. In *The Twelfth International Conference on Learning Representations*, 2024.
- Bao, G., Zhao, Y., He, J., and Zhang, Y. Glimpse: Enabling white-box methods to use proprietary models for zero-shot llm-generated text detection. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Basani, A. R. and Chen, P.-Y. Diversity boosts ai-generated text detection. *arXiv preprint arXiv:2509.18880*, 2025.
- BehnamGhader, P., Adlakha, V., Mosbach, M., Bahdanau, D., Chapados, N., and Reddy, S. Llm2vec: Large language models are secretly powerful text encoders. *arXiv preprint arXiv:2404.05961*, 2024.
- Biderman, S., Black, S., Gao, L., et al. Gpt-neox-20b: An open-source autoregressive language model. In *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*, 2023.
- Black, S., Gao, L., Wang, P., Leahy, C., and Biderman, S. Gpt-neo: Large scale autoregressive language modeling with mesh-tensorflow. <https://doi.org/10.5281/zenodo.5297715>, 2021. Zenodo.
- Chen, J., Zhu, X., Liu, T., Chen, Y., Xinhui, C., Yuan, Y., Leong, C. T., Li, Z., Tang, L., Zhang, L., et al. Imitate before detect: Aligning machine stylistic preference for machine-revised text detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 23559–23567, 2025.
- Chuang, Y.-N., Sarma, P. K., Gopalan, P., Boccio, J., Bolouki, S., Hu, X., and Zhou, H. Learning to route llms with confidence tokens. *arXiv preprint arXiv:2410.13284*, 2024.
- DeepSeek-AI. Deepseek llm: Let there be answers. <https://github.com/deepseek-ai/>
- DeepSeek-LLM, 2023. DeepSeek LLM 7B/67B Base and Chat models.
- Demszky, D., Yang, D., Yeager, D. S., Bryan, C. J., Clapper, M., Chandhok, S., Eichstaedt, J. C., Hecht, C., Jamieson, J., Johnson, M., et al. Using large language models in psychology. *Nature Reviews Psychology*, 2(11):688–701, 2023.
- Doshi, A. R. and Hauser, O. P. Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28):eadn5290, 2024.
- Dubey, A., Jauhri, A., Pandey, A., et al. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Fu, J., Guo, C.-L., and Li, C. Detectanyllm: Towards generalizable and robust detection of machine-generated text across domains and models. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 11229–11238, 2025a.
- Fu, Z., Chen, J., Sun, H., Yang, T., Li, R., and Zhang, Y. Fdllm: A text fingerprint detection method for llms in multi-language, multi-domain black-box environments. *arXiv e-prints*, pp. arXiv–2501, 2025b.
- Gehrmann, S., Strobelt, H., and Rush, A. GLTR: Statistical detection and visualization of generated text. In Costajussà, M. R. and Alfonseca, E. (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pp. 111–116, Florence, Italy, July 2019. Association for Computational Linguistics.
- Guo, H., Cheng, S., Jin, X., Zhang, Z., Zhang, K., Tao, G., Shen, G., and Zhang, X. Biscoper: AI-generated text detection by checking memorization of preceding tokens. *Advances in Neural Information Processing Systems*, 37: 104065–104090, 2024.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Hou, J., Zhao, J., Zhang, S., and Wang, P. A theoretical analysis of detecting large model-generated time series. *arXiv preprint arXiv:2511.07104*, 2025.
- Hu, Q. J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Keutzer, K., and Upadhyay, S. K. Router-bench: A benchmark for multi-llm routing system. *arXiv preprint arXiv:2403.12031*, 2024.
- Hu, X., Chen, P.-Y., and Ho, T.-Y. Radar: Robust AI-text detection via adversarial learning. *Advances in neural information processing systems*, 36:15077–15095, 2023.

- Huang, Z., Ling, G., Lin, Y., Chen, Y., Zhong, S., Wu, H., and Lin, L. Routereval: A comprehensive benchmark for routing llms to explore model-level scaling up in llms. *arXiv preprint arXiv:2503.10657*, 2025.
- Jitkrittum, W., Narasimhan, H., Rawat, A. S., Juneja, J., Wang, C., Wang, Z., Go, A., Lee, C.-Y., Shenoy, P., Panigrahy, R., et al. Universal model routing for efficient llm inference. *arXiv preprint arXiv:2502.08773*, 2025.
- Li, L., Wang, P., Ren, K., Sun, T., and Qiu, X. Origin tracing and detecting of llms. *arXiv preprint arXiv:2304.14072*, 2023.
- Li, Y., Li, Q., Cui, L., Bi, W., Wang, Z., Wang, L., Yang, L., Shi, S., and Zhang, Y. Mage: Machine-generated text detection in the wild. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 36–53, 2024.
- Liu, R., Huang, H., Xiao, X., and Wu, Z. Zero-shot detection of llm-generated text via implicit reward model. *Advances in Neural Information Processing Systems*, 38:167683–167709, 2026.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Ma, Y., Li, H., Zheng, X., Ling, F., Xiao, X., Wang, R., Wen, S., Chao, F., and Ji, R. Affinequant: Affine transformation quantization for large language models. In *International Conference on Learning Representations*, volume 2024, pp. 50932–50951, 2024.
- Ma, Y., Zheng, X., Xu, J., Xu, X., Ling, F., Zheng, X., Kuang, H., Li, H., Wang, X., Xiao, X., et al. Flow caching for autoregressive video generation. *arXiv preprint arXiv:2602.10825*, 2026.
- Milano, S., McGrane, J. A., and Leonelli, S. Large language models challenge the future of higher education. *Nature Machine Intelligence*, 5(4):333–334, 2023.
- Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D., and Finn, C. Detectgpt: Zero-shot machine-generated text detection using probability curvature. In *International Conference on Machine Learning*, pp. 24950–24962. PMLR, 2023.
- Narayan, S., Cohen, S. B., and Lapata, M. Don’t give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *ArXiv*, abs/1808.08745, 2018.
- OpenAI. Gpt-oss-20b. <https://huggingface.co/openai/gpt-oss-20b>, 2025. Open-source 20B parameter GPT model.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Solaiman, I., Brundage, M., Clark, J., Askell, A., Herbert-Voss, A., Wu, J., Radford, A., Krueger, G., Kim, J. W., Kreps, S., et al. Release strategies and the social impacts of language models. *arXiv preprint arXiv:1908.09203*, 2019.
- Song, Y., Yuan, Z., Zhang, S., Fang, Z., Yu, J., and Liu, F. Deep kernel relative test for machine-generated text detection. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Su, J., Zhuo, T., Wang, D., and Nakov, P. DetectLLM: Leveraging log rank information for zero-shot detection of machine-generated text. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 12395–12412, Singapore, December 2023a. Association for Computational Linguistics.
- Su, Z., Wu, X., Zhou, W., Ma, G., and Hu, S. Hc3 plus: A semantic-invariant human chatgpt comparison corpus. *arXiv preprint arXiv:2309.02731*, 2023b.
- Sun, K., Chen, S., Yao, T., Liu, H., Sun, X., Ding, S., and Ji, R. Diffusionfake: Enhancing generalization in deepfake detection via guided stable diffusion. *Advances in Neural Information Processing Systems*, 37:101474–101497, 2024.
- Sun, K., Chen, S., Yao, T., Zhou, Z., Ji, J., Sun, X., Lin, C.-W., and Ji, R. Towards general visual-linguistic face forgery detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 19576–19586, 2025a.
- Sun, K., Bao, G., Cui, H., and Zhang, Y. When ai settles down: Late-stage stability as a signature of ai-generated text detection. *arXiv preprint arXiv:2601.04833*, 2026.
- Sun, M., Yin, Y., Xu, Z., Kolter, J. Z., and Liu, Z. Idiosyncrasies in large language models. *arXiv preprint arXiv:2502.12150*, 2025b.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wang, B. and Komatsuzaki, A. Gpt-j-6b: A 6 billion parameter autoregressive language model. <https://github.com/kingoflolz/mesh-transformer-jax>, 2021. Accessed: 2025-11-29.

- Wu, C., Cheung, Y.-m., Han, B., and Lian, D. Advancing machine-generated text detection from an easy to hard supervision perspective. *Advances in Neural Information Processing Systems*, 38:150210–150258, 2026.
- Wu, J., Yang, S., Zhan, R., Yuan, Y., Chao, L. S., and Wong, D. F. A survey on LLM-generated text detection: Necessity, methods, and future directions. *Computational Linguistics*, pp. 1–66, 2025.
- Xu, Y., Wang, Y., An, H., Liu, Z., and Li, Y. Detecting subtle differences between human and model languages using spectrum of relative likelihood. *arXiv preprint arXiv:2406.19874*, 2024a.
- Xu, Y., Wang, Y., Bi, Y., Cao, H., Lin, Z., Zhao, Y., and Wu, F. Training-free llm-generated text detection by mining token probability sequences. *arXiv preprint arXiv:2410.06072*, 2024b.
- Yang, X., Pan, L., Zhao, X., Chen, H., Petzold, L. R., Wang, W. Y., and Cheng, W. A survey on detection of LLMs-generated content. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 9786–9805, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- Yu, X., Qi, Y., Chen, K., Chen, G., Yang, X., Zhu, P., Shang, X., Zhang, W., and Yu, N. DPIC: Decoupling prompt and intrinsic characteristics for llm generated text detection. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 16194–16212. Curran Associates, Inc., 2024.
- Yu, X., Yu, Y., Liu, D., Chen, K., Zhang, W., Yu, N., and Shao, J. Evobench: Towards real-world llm-generated text detection benchmarking for evolving large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 14605–14620, 2025.
- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., Mihaylov, T., Ott, M., Shleifer, S., Shuster, K., Simig, D., Koura, P. S., Sridhar, A., Wang, T., and Zettlemoyer, L. OPT: Open pre-trained transformer language models, 2022.
- Zhang, S., Song, Y., Yang, J., Li, Y., Han, B., and Tan, M. Detecting machine-generated texts by multi-population aware optimization for maximum mean discrepancy. In *The Twelfth International Conference on Learning Representations*, 2024.
- Zhou, Z., Sun, K., Chen, Z., Kuang, H., Sun, X., and Ji, R. Stealthdiffusion: Towards evading diffusion forensic detection through diffusion model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 3627–3636, 2024.
- Zhou, Z., Luo, Y., Wu, Y., Sun, K., Ji, J., Yan, K., Ding, S., Sun, X., Wu, Y., and Ji, R. Aigi-holmes: Towards explainable and generalizable ai-generated image detection via multimodal large language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 18746–18758, 2025.
- Zhu, X., Ren, Y., Fang, F., Tan, Q., Wang, S., and Cao, Y. Dna-detectllm: Unveiling ai-generated text via a dna-inspired mutation-repair paradigm. *Advances in Neural Information Processing Systems*, 38:1745–1768, 2026.

A. Theoretical Analysis

This section provides the complete proof of Proposition 3.1, establishing that the performance gap in zero-shot detection is fundamentally bounded by the distributional divergence between source and surrogate models.

A.1. Preliminaries

We introduce the necessary background on probability divergences.

Definition A.1 (Total Variation Distance). For two probability distributions P and Q defined on a measurable space $(\mathcal{X}, \mathcal{F})$, the total variation distance is

$$d_{TV}(P, Q) = \sup_{A \in \mathcal{F}} |P(A) - Q(A)| = \frac{1}{2} \int_{\mathcal{X}} |p(x) - q(x)| dx \quad (13)$$

where p and q denote the probability density functions of P and Q , respectively.

Definition A.2 (Kullback-Leibler Divergence). For two probability distributions P and Q with P absolutely continuous with respect to Q , the KL divergence is

$$D_{KL}(P \| Q) = \int_{\mathcal{X}} p(x) \log \frac{p(x)}{q(x)} dx \quad (14)$$

Lemma A.3 (Pinsker's Inequality). For any two probability distributions P and Q :

$$d_{TV}(P, Q) \leq \sqrt{\frac{1}{2} D_{KL}(P \| Q)} \quad (15)$$

This classical result connects the total variation distance, which is directly useful for bounding expectation differences, to the KL divergence, which has information-theoretic interpretations and is often more tractable to estimate.

A.2. Proof of the Mismatch Risk Bound

Proof. Let $T(x) : \mathcal{X} \rightarrow \mathbb{R}$ be a zero-shot scoring function with $|T(x)| \leq B$ for all $x \in \mathcal{X}$. Let P_{src} denote the source distribution and P_{sur} denote the surrogate distribution, with corresponding density functions p_{src} and p_{sur} .

The optimal detection signal is

$$\mu^* = \mathbb{E}_{x \sim P_{src}}[T(x)] = \int_{\mathcal{X}} T(x) p_{src}(x) dx \quad (16)$$

The proxy signal obtained using the surrogate is

$$\mu_{proxy} = \mathbb{E}_{x \sim P_{sur}}[T(x)] = \int_{\mathcal{X}} T(x) p_{sur}(x) dx \quad (17)$$

Step 1: Express the gap as an integral. The performance gap can be written as

$$|\mu^* - \mu_{proxy}| = \left| \int_{\mathcal{X}} T(x) p_{src}(x) dx - \int_{\mathcal{X}} T(x) p_{sur}(x) dx \right| \quad (18)$$

$$= \left| \int_{\mathcal{X}} T(x) (p_{src}(x) - p_{sur}(x)) dx \right| \quad (19)$$

Step 2: Apply the boundedness condition. Since $|T(x)| \leq B$, we have

$$\left| \int_{\mathcal{X}} T(x) (p_{src}(x) - p_{sur}(x)) dx \right| \leq \int_{\mathcal{X}} |T(x)| \cdot |p_{src}(x) - p_{sur}(x)| dx \quad (20)$$

$$\leq B \int_{\mathcal{X}} |p_{src}(x) - p_{sur}(x)| dx \quad (21)$$

Step 3: Relate to total variation distance. By definition:

$$\int_{\mathcal{X}} |p_{src}(x) - p_{sur}(x)| dx = 2 \cdot d_{TV}(P_{src}, P_{sur}) \quad (22)$$

Therefore:

$$|\mu^* - \mu_{proxy}| \leq 2B \cdot d_{TV}(P_{src}, P_{sur}) \quad (23)$$

Step 4: Apply Pinsker's inequality. Using Lemma A.3:

$$|\mu^* - \mu_{proxy}| \leq 2B \cdot d_{TV}(P_{src}, P_{sur}) \quad (24)$$

$$\leq 2B \cdot \sqrt{\frac{1}{2} D_{KL}(P_{src} \| P_{sur})} \quad (25)$$

$$= B \cdot \sqrt{2 D_{KL}(P_{src} \| P_{sur})} \quad (26)$$

This completes the proof. $\square$

A.3. Implications for Zero-Shot Detection

The Mismatch Risk Bound has several important implications for zero-shot LLM detection.

Monotonicity in Divergence. The bound is monotonically increasing in $D_{KL}(P_{src} \| P_{sur})$. As the distributional divergence between source and surrogate increases, the potential performance degradation also increases. This theoretical prediction aligns precisely with our empirical observations in Table 1, where cross-model performance degrades significantly compared to the matched setting.

Optimality of Routing. Given a pool of candidate surrogates $\{M_1, M_2, \dots, M_K\}$ with corresponding distributions $\{P_{M_1}, P_{M_2}, \dots, P_{M_K}\}$, the bound suggests that the optimal surrogate for a source P_{src} is

$$M^* = \arg \min_{k \in [K]} D_{KL}(P_{src} \| P_{M_k}) \quad (27)$$

This minimization tightens the error bound and recovers the best possible detection performance within the surrogate pool, directly motivating our routing approach.

Architectural Affinity. The structured patterns observed in Figure 2, where models from the same family exhibit higher mutual affinity, can be explained through this framework. Models sharing architectural design choices and pre-training data are likely to have smaller KL divergence, leading to tighter bounds and better cross-detection performance.

Variance in Black-box Settings. The extreme performance variance observed in Figure 1 corresponds to the varying values of $D_{KL}(P_{src} \| P_{M_k})$ across different surrogates M_k . For a fixed black-box source, some surrogates happen to have distributions closer to the source, resulting in high detection AUC, while others have large divergence, leading to near-random performance.

A.4. Extension to Finite Samples

In practice, we work with finite samples rather than true distributions. The bound can be extended to account for estimation error using standard concentration inequalities.

Corollary A.4 (Finite Sample Bound). *Let $\hat{\mu}^*$ and $\hat{\mu}_{proxy}$ be empirical estimates of μ^* and μ_{proxy} based on n i.i.d. samples. Then with probability at least $1 - \delta$:*

$$|\hat{\mu}^* - \hat{\mu}_{proxy}| \leq B \cdot \sqrt{2 D_{KL}(P_{src} \| P_{sur})} + 2B \sqrt{\frac{\log(2/\delta)}{2n}} \quad (28)$$

The additional term captures the statistical uncertainty from finite sampling, which vanishes as $n \rightarrow \infty$.

Table 6. Detection performance on MAGE across eight model families. AUROC is reported in percentage. The best result in each column is highlighted in bold.

Method	OpenAI	GLM	Human-Para	FLAN	LLaMA	BigScience	OPT	EleutherAI	Avg
Likelihood	89.12	91.30	60.61	30.98	92.96	37.51	31.21	0.26	56.81
Entropy	23.08	17.98	62.83	66.65	20.30	66.22	76.39	81.76	51.90
Rank	73.74	83.12	53.58	34.93	81.74	45.49	52.75	46.63	58.99
LogRank	87.05	90.28	53.81	31.56	91.69	37.38	32.39	23.10	55.91
LLR	72.90	74.24	36.21	36.17	78.09	39.97	40.17	36.52	51.78
Fast-DetectGPT	76.40	80.41	61.92	76.29	67.35	72.40	64.61	58.12	69.69
Lastde	74.47	83.87	62.96	66.35	72.01	64.63	72.35	69.07	70.71
FourierGPT	59.31	84.88	68.32	4.72	84.39	39.60	57.55	63.97	60.34
Diveye	87.89	92.61	62.10	34.65	93.80	52.00	67.60	66.11	69.60
UCE	18.81	20.02	70.60	67.05	23.48	62.86	82.81	87.73	54.17
DetectRouter	93.24	90.37	88.52	76.66	72.55	74.65	67.56	59.81	77.92

A.5. Connection to Information Theory

The appearance of KL divergence in the bound is not coincidental. From an information-theoretic perspective, $D_{KL}(P_{src}||P_{sur})$ quantifies the information lost when using P_{sur} to approximate P_{src} . In the context of LLM detection, this information loss directly translates to degraded detection performance, as the surrogate model fails to capture the distinctive statistical patterns of the source.

This connection suggests that effective routing can be viewed as an information-theoretic channel matching problem: we seek to route each input to the surrogate that minimizes the information loss, thereby maximizing the fidelity of the detection signal.

B. Additional Experimental Results

This section provides comprehensive experimental results that complement the main paper.

B.1. Full MAGE Benchmark Results

Table 6 presents the complete per-family breakdown on the MAGE benchmark. The results reveal substantial heterogeneity in baseline performance across model families. For instance, Likelihood achieves 92.96% on LLaMA-family text but only 0.26% on EleutherAI, reflecting severe surrogate-source mismatch when architectures differ substantially. Our routing mechanism effectively mitigates this variance by dynamically selecting appropriate surrogates.

B.2. Full Routing Enhancement Results

Tables 7 and 8 present the complete per-family results for applying DetectRouter to each zero-shot detection method on EvoBench and MAGE, respectively.

B.3. Supervised Baseline Results

We evaluate supervised detection using each surrogate model’s representations trained on Stage 2 data for binary classification. Tables 9 and 10 present the complete results on EvoBench and MAGE, respectively. Performance varies substantially across surrogates, demonstrating that supervised approaches are highly sensitive to the choice of representation model. For instance, DeepSeek-7B achieves 79.30% on EvoBench while Falcon-7B only reaches 48.27%, highlighting the importance of surrogate selection even in supervised settings. DetectRouter consistently outperforms all single-surrogate supervised baselines.

Table 7. Routing enhancement on EvoBench. Per-family AUROC is reported in percentage for each detection method with and without routing. Δ denotes relative improvement from applying DetectRouter.

Method	LLaMA3	LLaMA2	Claude	GPT-4o	Gemini	GPT-4	Qwen	Avg	Δ
Likelihood	88.84	61.68	79.70	77.80	78.89	77.80	77.40	77.44	+5.4%
Likelihood+Ours	90.58	66.12	84.56	85.92	84.20	84.44	75.31	81.59	
Entropy	28.85	57.58	33.61	35.43	38.34	38.10	30.34	37.46	+139.4%
Entropy+Ours	94.78	80.20	92.75	92.24	92.36	92.59	83.01	89.70	
Rank	68.49	58.38	64.50	64.64	64.46	63.43	55.06	62.71	+17.0%
Rank+Ours	78.13	69.24	75.01	76.35	74.34	75.06	65.48	73.37	
LogRank	86.47	59.99	77.37	76.80	75.99	74.87	76.10	75.37	+9.5%
LogRank+Ours	90.26	70.31	83.82	86.07	84.35	84.47	78.67	82.56	
LLR	66.46	49.61	61.73	60.57	59.12	58.43	63.08	59.86	+33.2%
LLR+Ours	85.97	71.96	80.05	81.57	81.46	80.66	76.68	79.76	
Fast-DetectGPT	94.17	80.27	81.57	74.89	80.48	75.58	80.10	81.01	+12.1%
Fast-DetectGPT+Ours	95.66	80.82	94.22	94.29	93.79	94.16	83.01	90.85	

Table 8. Routing enhancement on MAGE. Per-family AUROC is reported in percentage for each detection method with and without routing. Δ denotes relative improvement from applying DetectRouter.

Method	OpenAI	GLM	Human-Para	FLAN	LLaMA	BigScience	OPT	EleutherAI	Avg	Δ
Likelihood	89.12	91.30	60.61	30.98	92.96	37.51	31.21	0.26	56.81	+14.7%
Likelihood+Ours	91.67	85.68	83.46	54.41	72.62	55.02	47.54	30.85	65.16	
Entropy	23.08	17.98	62.83	66.65	20.30	66.22	76.39	81.76	51.90	+26.5%
Entropy+Ours	76.16	42.35	86.80	68.37	33.29	68.62	73.12	76.37	65.64	
Rank	73.74	83.12	53.58	34.93	81.74	45.49	52.75	46.63	58.99	+15.6%
Rank+Ours	83.54	85.74	79.94	53.48	74.40	58.91	61.61	47.90	68.19	
LogRank	87.05	90.28	53.81	31.56	91.69	37.38	32.39	23.10	55.91	+22.8%
LogRank+Ours	91.94	90.39	80.81	54.65	79.04	58.98	54.19	39.07	68.63	
LLR	72.90	74.24	36.21	36.17	78.09	39.97	40.17	36.52	51.78	+35.4%
LLR+Ours	89.40	90.17	72.46	59.63	81.28	61.35	59.98	46.77	70.13	
Fast-DetectGPT	76.40	80.41	61.92	76.29	67.35	72.40	64.61	58.12	69.69	+11.8%
Fast-DetectGPT+Ours	93.24	90.37	88.52	76.66	72.55	74.65	67.56	59.81	77.92	

B.4. Prototype Space Geometry

To verify the quality of the learned metric space, we visualize test set embeddings using t-SNE in Figure 7. The visualization reveals two critical geometric properties. First, texts generated by different model families form distinct, well-separated clusters, validating that our contrastive learning objective captures model-specific artifacts. Second, the learned prototypes are accurately positioned at the centroids of their respective source clusters. GPT-J and GPT-Neo clusters appear adjacent, reflecting their shared architectural lineage, yet remain distinguishable. This coherent structure confirms that DetectRouter learns a semantic manifold where proximity correlates with detector affinity, enabling robust generalization.

B.5. What DetectRouter Learns: Beyond Max-Selection

A natural reading of Sections 4.3–4.4 might suggest that DetectRouter merely learns a soft approximation to max-selection: training from softmax-normalized surrogate scores and then taking a top-1 detector at inference. We clarify why this reading is incorrect, both conceptually and empirically.

Conceptual Distinction. Max-selection operates entirely in *score space*, taking $\max_k s_k(x)$ across all N surrogates per input. DetectRouter operates in *representation space*: it first infers source affinity via nearest-prototype matching, then scores with *only* the selected surrogate. The Stage 2 softmax-over-scores teaches the router which detector family each

Table 9. Supervised detection results on EvoBench using different surrogate representations trained on Stage 2 data. AUROC is reported in percentage.

Surrogate	LLaMA3	LLaMA2	Claude	GPT-4o	Gemini	GPT-4	Qwen	Avg
LLaMA3-8B	74.96	66.88	73.38	77.66	62.07	76.16	66.49	71.09
DeepSeek-7B	85.71	68.16	83.89	84.64	74.05	84.85	73.81	79.30
Falcon-7B	43.47	56.67	42.32	46.58	50.48	47.86	50.54	48.27
GPT2-XL	61.16	61.81	65.08	62.51	74.42	62.08	61.03	64.01
GPT-J-6B	73.92	61.72	72.85	73.74	54.55	72.66	68.78	68.32
GPT-Neo-2.7B	69.42	62.62	73.42	75.20	69.41	73.81	66.46	70.05
OPT-2.7B	72.79	64.32	68.75	72.12	68.17	72.16	65.64	69.14
Qwen-3B	82.22	64.96	81.18	80.59	76.22	80.81	69.56	76.51
GPT-oss-20B	50.07	57.80	47.90	51.15	50.72	51.93	60.73	52.90
GPT-NeoX-20B	57.93	56.44	58.39	57.56	57.37	57.74	55.89	57.33
DetectRouter	95.66	80.82	94.22	94.29	93.79	94.16	83.01	90.85

Table 10. Supervised detection results on MAGE using different surrogate representations trained on Stage 2 data. AUROC is reported in percentage.

Surrogate	OpenAI	GLM	Human-Para	FLAN	LLaMA	BigScience	OPT	EleutherAI	Avg
LLaMA3-8B	83.11	81.82	82.15	75.26	72.13	74.51	64.41	64.15	74.69
DeepSeek-7B	89.38	82.26	86.84	74.23	71.53	75.80	71.21	69.78	77.63
Falcon-7B	48.88	77.60	72.32	39.83	75.61	47.72	59.56	61.87	60.42
GPT2-XL	52.34	51.10	49.01	48.28	50.41	46.60	39.82	34.43	46.50
GPT-J-6B	81.61	77.60	78.14	63.61	70.17	62.57	50.07	49.31	66.64
GPT-Neo-2.7B	60.85	75.08	60.78	48.37	71.03	53.22	51.28	48.25	58.61
OPT-2.7B	78.13	74.96	74.66	67.43	67.32	62.51	52.10	48.35	65.68
Qwen-3B	88.66	81.14	80.99	68.58	68.20	65.34	62.09	56.71	71.46
GPT-oss-20B	53.76	61.59	63.87	44.68	59.36	51.26	56.76	63.29	56.82
GPT-NeoX-20B	71.42	68.24	69.67	64.51	65.65	64.96	52.24	52.18	63.61
DetectRouter	93.24	90.37	88.52	76.66	72.55	74.65	67.56	59.81	77.92

input is most compatible with—it learns detector affinity, not score extremization. Critically, since human text has no true source LLM, taking the maximum over N surrogates systematically inflates human scores via a multiple-testing effect, compressing the human–machine margin. Routing avoids this because each input is scored by exactly one selected detector.

Empirical Distinction. Table 11 compares DetectRouter against the max-selection baseline. Max-selection (72.33% on EvoBench) falls 8.68 points *below* even the fixed LLaMA3-8B baseline, confirming that score-level aggregation actively harms detection. DetectRouter outperforms max-selection by 18.52 points on EvoBench, decisively ruling out the interpretation that routing is implicit max-pooling. We additionally report an oracle baseline that selects the best surrogate per sample using ground-truth source labels: DetectRouter closes most of the fixed-to-oracle gap (about 72% on EvoBench, 76% on MAGE) without any access to source identity at inference time.

Routing Behavior: Human vs. Machine Text. The asymmetry between human and machine text further distinguishes routing from max-selection. Table 12 reports routing statistics over the full EvoBench test set (32,993 pairs). Human texts concentrate sharply on a single surrogate (86.1% routed to top-1, entropy 0.78 bits), reflecting that human writing has no model-specific fingerprint and the router consistently maps it to a stable “human-like” region of the metric space. Machine texts, in contrast, spread across the pool (49.1% to top-1, entropy 2.12 bits), exhibiting source-specific routing preferences. At the equal-error-rate threshold, this asymmetry translates into a false positive rate of 6.7% for DetectRouter versus 74.2% for max-selection. Max-selection, by construction, cannot produce this asymmetry: it inflates scores symmetrically for any

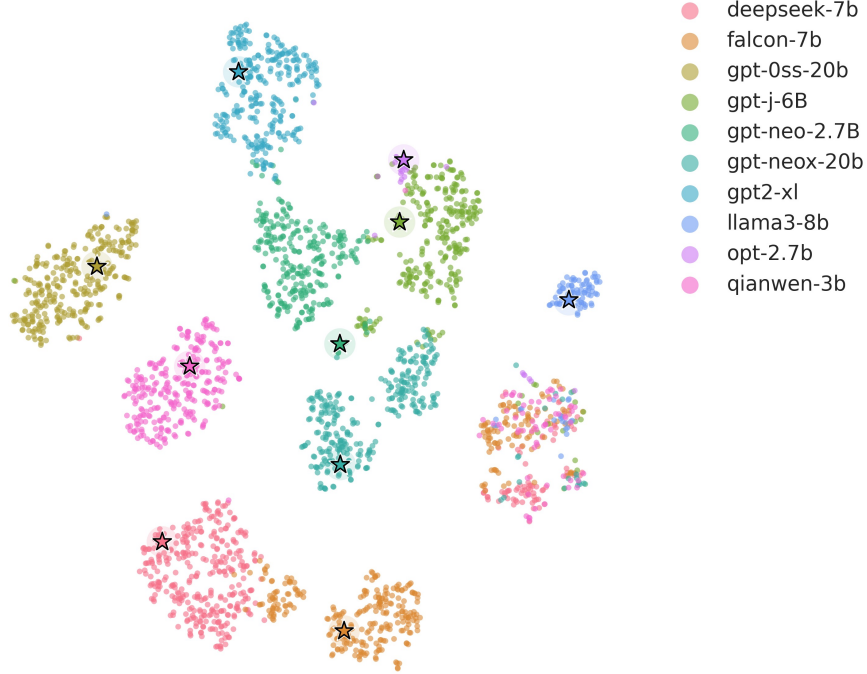


Figure 7. T-SNE projection of the embedding space showing text samples from different source models alongside their corresponding prototypes.

Table 11. Comparison with max-selection and oracle baselines. Max-selection takes the highest oriented score across the surrogate pool; Oracle selects the best surrogate per sample with ground-truth labels. AUROC in percentage.

Method	EvoBench	MAGE
Max-selection (per sample)	72.33	70.35
Fixed (LLaMA3-8B)	81.01	69.69
DetectRouter	90.85	77.92
Oracle (per-sample best)	94.76	80.47

input, including human text. The fact that DetectRouter *can* produce it confirms the router learns text–detector affinity, not score-maximizing behavior.

B.6. Generalization to Unseen Model Families

A practical concern is whether DetectRouter generalizes to source families absent from the Stage 2 training distribution. We address this from two angles.

Held-out Proprietary Families. Our main EvoBench evaluation already includes families with *no* corresponding surrogate in the pool: GPT-4o, Claude, and Gemini are entirely proprietary. As shown in Table 2, DetectRouter achieves 94.22% on Claude, 94.29% on GPT-4o, and 94.16% on GPT-4, indicating that the prototype space captures distributional fingerprints that transfer beyond the surrogate pool.

Leave-One-Family-Out. To stress-test this further, we conduct leave-one-family-out experiments by removing one open-source family from Stage 2 training at a time and re-evaluating. Table 13 shows graceful degradation of approximately 1.6% on average, with all variants still substantially exceeding the strongest fixed-surrogate baseline (81.01% on EvoBench). This robustness stems from Stage 1’s metric-space learning of distributional fingerprints that transfer across families with shared architectural lineage or training corpora.

Table 12. Routing behavior on human vs. machine text in EvoBench. Top-1/Top-2 concentration measures the fraction of inputs routed to the most/two most preferred surrogates; entropy is computed over the routing distribution.

Metric	Human	Machine
Routed to top-1 surrogate	86.1%	49.1%
Routed to top-2 surrogates	96.1%	75.4%
Routing entropy (bits)	0.78	2.12
Cross-surrogate score std	0.52	0.68

Table 13. Leave-one-family-out generalization. We remove one open-source family from Stage 2 training and re-evaluate. AUROC in percentage.

Removed Family	EvoBench	MAGE
None (full pool)	90.85	77.92
DeepSeek	89.12	76.84
Qwen	88.47	76.21
Falcon	90.31	77.45

B.7. Incremental Pool Expansion

A practical concern for deploying DetectRouter is the cost of expanding the surrogate pool as new LLMs emerge. We verify that pool expansion is lightweight via an incremental learning experiment. Starting from the 9-model pool without DeepSeek (89.12% on EvoBench, per Table 13), we freeze all existing prototypes, introduce new DeepSeek prototypes, and fine-tune using only 10% of DeepSeek’s Stage 2 scores. Performance recovers to 90.41%, close to the full 10-model result of 90.85%.

This confirms that integrating a new surrogate requires scoring only a small fraction of the dataset with the new model—consistent with the data-efficiency analysis in Figure 6, which shows 10% of Stage 2 data suffices to reach 89.06% on EvoBench. Combined with the linear $N \times K$ distance-computation cost at inference, this makes DetectRouter practical for pools of dozens to hundreds of models.

B.8. Hyperparameter Sensitivity

Table 14 reports detection performance across variations in the three weighting hyperparameters of our composite Stage 2 objective: λ_{sep} (inter-class separation), λ_{anc} (prototype anchoring), and λ_{norm} (prototype norm regularization). Variation across λ_{sep} and λ_{anc} remains within $\pm 0.8\%$ on EvoBench. Sweeping λ_{norm} from 10^{-5} to 10^{-3} causes less than 0.3% change. All three are low-sensitivity, indicating that the routing framework is robust to hyperparameter choice.

B.9. Distance Metric Ablation

Table 15 compares distance metrics used in the routing step. L2 distance on cosine-normalized features outperforms both unnormalized L2 and pure cosine similarity. Normalization prevents magnitude-dominated distances and focuses on directional similarity between embeddings, which is essential when prototypes from different model families may exhibit varying norms after training.

B.10. Multilingual Generalization

While our main experiments focus on English text, real-world disinformation risks are inherently multilingual. We evaluate DetectRouter on the M4 benchmark, which covers 8 languages with 120 human and 120 machine samples per language (1,920 samples total). Table 16 shows that DetectRouter outperforms the fixed Fast-DetectGPT baseline by 8.2 percentage points on average, with the most striking gain on Chinese ($0.526 \rightarrow 0.941$). Fixed Fast-DetectGPT collapses on Chinese because LLaMA3-8B’s tokenization is suboptimal for CJK scripts, while routing selects surrogates with better CJK coverage such as Qwen-3B. This is precisely the mismatch scenario that routing is designed to resolve. Performance is consistently strong across Semitic, Slavic, Sino-Tibetan, Germanic, Austronesian, and Indo-Aryan languages, confirming that the prototype space captures language-agnostic distributional fingerprints.

Table 14. Hyperparameter sensitivity. Average EvoBench AUROC (%) across λ_{sep} (rows) and λ_{anc} (columns). The default configuration ($\lambda_{\text{sep}} = 10^{-3}$, $\lambda_{\text{anc}} = 1.0$) is shaded.

$\lambda_{\text{sep}} \backslash \lambda_{\text{anc}}$	0.1	1.0	10.0
10^{-4}	89.93	90.17	89.64
10^{-3}	90.24	90.85	90.41
10^{-2}	89.71	90.08	89.32

Table 15. Ablation on the routing distance metric. AUROC in percentage.

Distance Metric	EvoBench	MAGE
L2 (unnormalized)	87.63	75.18
Cosine similarity	90.12	77.54
L2 + normalization (ours)	90.85	77.92

B.11. Adversarial Robustness

We evaluate adversarial robustness on the RAID benchmark across five attack types: whitespace insertion, upper/lower case flipping, synonym substitution, misspelling, and paraphrase. Each attack contains 3,547 labeled pairs. Table 17 shows DetectRouter improves average AUROC by 9.6 percentage points across all attacks. Surface-level perturbations degrade fixed curvature estimates, but routing selects surrogates whose tokenization is less sensitive to each perturbation type. On the hardest paraphrase attack, DetectRouter still gains 6.6 points by adaptively selecting the surrogate whose distribution remains closest to the paraphrased source.

We note that routing robustness is orthogonal to detection criterion design, so combining with more robust criteria such as LLR would yield further gains. A dedicated adversary targeting the routing mechanism itself is an interesting direction for future work; such attacks would require knowledge of internal representations and are much harder to mount than generic text perturbations.

B.12. Mixed-Region Routing Analysis

The t-SNE visualization in Figure 7 shows a mixed cluster in the lower-right region where samples from multiple model families overlap. We analyze routing behavior in this region to understand the framework’s graceful degradation properties. We find that the overlap is driven primarily by task-induced similarity: samples in this region predominantly come from constrained generation tasks (Polish and Rewrite), where strong constraints push different models toward similar surface patterns.

Table 18 reports routing metrics for this region. Routing confidence drops, as expected, with entropy increasing from 0.64 to 2.03 bits. AUROC degrades by only 6.3 points ($92.4 \rightarrow 86.1$), however, because the top-ranked surrogates in this region perform comparably: when task constraints compress inter-model differences, multiple surrogates achieve similar detection quality, so imperfect routing still yields a reasonable detector. This represents graceful degradation rather than a failure mode.

B.13. Comparison with MIRAGE Official Supervised Baselines

MIRAGE provides official supervised detection checkpoints. We compare DetectRouter against these checkpoints under the same evaluation setting in Table 19. The MIRAGE official checkpoint outperforms our own supervised baseline on EvoBench (74.33 vs. 71.09), demonstrating the quality of its training framework. However, it struggles on older families in MAGE (e.g., EleutherAI at 26.83%, OPT at 38.14%), reflecting the supervised approach’s sensitivity to training distribution. DetectRouter outperforms the MIRAGE official checkpoint by 16.52 points on EvoBench and 7.81 points on MAGE.

We additionally evaluate whether the gain comes from data volume by training a supervised classifier on both Stage 1 and Stage 2 data. Even with this expanded training set, the supervised baseline reaches only 72.53% on EvoBench—still 18.32 points below DetectRouter. The gain therefore stems from learned text-detector affinity patterns, not from training data scale.

Table 16. Multilingual detection on the M4 benchmark across 8 languages. AUROC reported as decimal. AR: Arabic, BG: Bulgarian, ZH: Chinese, EN: English, DE: German, ID: Indonesian, RU: Russian, UR: Urdu.

Method	AR	BG	ZH	EN	DE	ID	RU	UR	Avg
F-DetectGPT (Fixed)	0.914	0.909	0.526	0.969	0.835	0.957	0.598	0.998	0.838
DetectRouter	0.952	0.938	0.941	0.981	0.872	0.978	0.701	0.999	0.920

Table 17. Adversarial robustness on the RAID benchmark across five attack types. AUROC reported as decimal.

Method	Whitespace	Upper/Lower	Synonym	Misspell	Paraphrase	Avg
F-DetectGPT (Fixed)	0.582	0.570	0.663	0.519	0.628	0.592
DetectRouter	0.683	0.672	0.752	0.641	0.694	0.688

B.14. Computational Cost Breakdown

For practical deployment, we report a complete cost breakdown.

Inference Cost. The router adds a single encoding pass plus $N \times K$ prototype distance computations (with N surrogates and $K = 10$ prototypes each). Distance computation costs $\sim 0.01\text{ms}$ regardless of encoder choice. With RoBERTa-base (88.34% on EvoBench), routing latency is 4.87ms per sample—negligible compared to the surrogate’s own scoring pass. With LLM2Vec-LLaMA3-8B (90.85% on EvoBench), latency rises to 166.2ms per sample. Users can choose the encoder based on the throughput/accuracy trade-off (Table 4).

Training Cost. Stage 2’s dominant cost is computing detection scores across the surrogate pool. On MIRAGE with 10 surrogates, this requires approximately 15 GPU-hours on a single A100, fully parallelizable. Stage 2 training itself takes approximately 8 GPU-hours. Both costs are modest compared to training the surrogate LLMs themselves, and the score-computation step is a one-time preprocessing operation. As shown in our incremental learning experiment (Appendix B.7), expanding the pool requires scoring only a small fraction of the dataset with new surrogates.

C. Additional Analysis Visualizations

This section provides additional visualizations that complement the analysis in Section 3, illustrating the fundamental challenges that motivate our routing approach.

C.1. Performance Gap Between Matched and Mismatched Settings.

Figure 8 visualizes the performance degradation when moving from white-box to cross-model detection settings. For each of the five zero-shot detection methods, we compare the AUC achieved when the surrogate matches the source model versus when the surrogate is mismatched. The cyan bars represent white-box performance where the detector has access to the true source model, while the pink bars show the average cross-model performance with error bars indicating variance across different surrogate choices. The results reveal a consistent and substantial performance drop across all methods. Likelihood-based detection drops from 0.81 to 0.47, falling below random chance. LogRank exhibits the most dramatic degradation, plummeting from 0.85 to 0.50. Even Fast-DetectGPT, the most robust method, experiences a 20-point decline from 0.94 to 0.74. The large error bars in the cross-model setting further highlight the instability of fixed-surrogate approaches, where performance varies dramatically depending on the arbitrary choice of surrogate.

C.2. Variance in Black-Box Detection Performance.

Figure 9 provides a detailed examination of detection performance variance when targeting black-box source models. Using the Fast-DetectGPT criterion on the MIRAGE benchmark, we evaluate each of our ten open-source surrogates against text generated by nine proprietary black-box model families: Claude, DeepSeek, Doubao, GPT, Llama, Qwen, Gemini, Grok, and Moonshot. Each column represents a different black-box source, with individual data points showing the AUC achieved by each surrogate. The box plots summarize the distribution of surrogate performance for each source. Two critical observations emerge from this visualization. First, optimal surrogates exist for every black-box source, with maximum AUC values ranging from 0.92 for Claude to 1.00 for Llama, demonstrating that effective detection is achievable given the right

Table 18. Routing metrics in well-separated vs. mixed regions of the prototype space.

Region	Routing Acc.	Entropy (bits)	AUROC
Well-separated	93.7%	0.64	92.4
Mixed region	71.8%	2.03	86.1

Table 19. Comparison with MIRAGE official supervised baselines. AUROC in percentage.

Method	EvoBench	MAGE
MIRAGE official (Qwen2-0.5B)	74.33	70.11
Our supervised (LLaMA3, Stage 2 only)	71.09	74.69
Our supervised (LLaMA3, Stage 1 + 2)	72.53	74.03
DetectRouter	90.85	77.92

surrogate selection. Second, the performance variance is extreme, with AUC spreads exceeding 0.70 for several sources. DeepSeek exhibits the widest range from 0.15 to 0.99, while GPT spans from 0.30 to 0.96. This variance confirms that surrogate selection is not merely an optimization but a fundamental requirement for reliable detection. A random surrogate choice frequently yields near-random performance, as indicated by values approaching the 0.5 baseline shown by the red dashed line.

C.3. Empirical Validation of the Mismatch Risk Bound

While Proposition 3.1 bounds the magnitude of score distortion in expectation, an end-to-end connection to AUROC and to our prototype-based routing requires empirical validation. We provide such validation from two complementary angles.

Prototype Distance vs. AUROC (Indirect Validation). The Mismatch Risk Bound predicts that larger source–surrogate divergence yields lower detection AUROC. To test this, we compute the Spearman rank correlation between prototype distance and AUROC across all 72 off-diagonal source–surrogate pairs in the 9×9 cross-evaluation matrix (Figure 2). The correlation is $\rho = -0.76$ with $p < 0.001$, consistent with the theoretical prediction that geometric proximity in the learned prototype space reflects detection-relevant alignment.

Prototype Distance vs. Token-level KL (Direct Validation). To verify that prototype distances reflect *true* distributional divergence—not merely empirical detection scores—we directly estimate KL divergence between model pairs. For each of the 72 source–surrogate pairs, we run both models on a shared held-out corpus and compute the average per-token KL divergence over output distributions. The Pearson correlation between these token-level KL estimates and prototype distances is $r = 0.72$ with $p < 0.001$, confirming that the learned metric space captures intrinsic distributional structure rather than incidental score-fitting artifacts.

Together, these results substantiate the chain prototype distance $\Leftrightarrow$ KL divergence $\Rightarrow$ detection performance that underpins DetectRouter’s design.

C.4. Alignment Objective Ablation

Section 4.3 aligns the predicted affinity distribution with the observed score distribution via KL divergence. To verify that this distributional formulation, rather than absolute-score regression, is responsible for the gains, we replace the Stage 2 alignment objective with alternative losses while keeping all other components fixed. Table 20 shows that KL outperforms L_1 , L_2 , and cosine alternatives by 3.4–4.3 points on EvoBench. This aligns with the per-detector mismatch principle of Proposition 3.1: matching the *shape* of the affinity distribution preserves relative ordering across detectors, whereas regressing absolute scores is unstable due to detector-specific scale differences.

D. Experimental Setup Details

This section provides comprehensive details on training data construction, model configurations, and evaluation protocols.

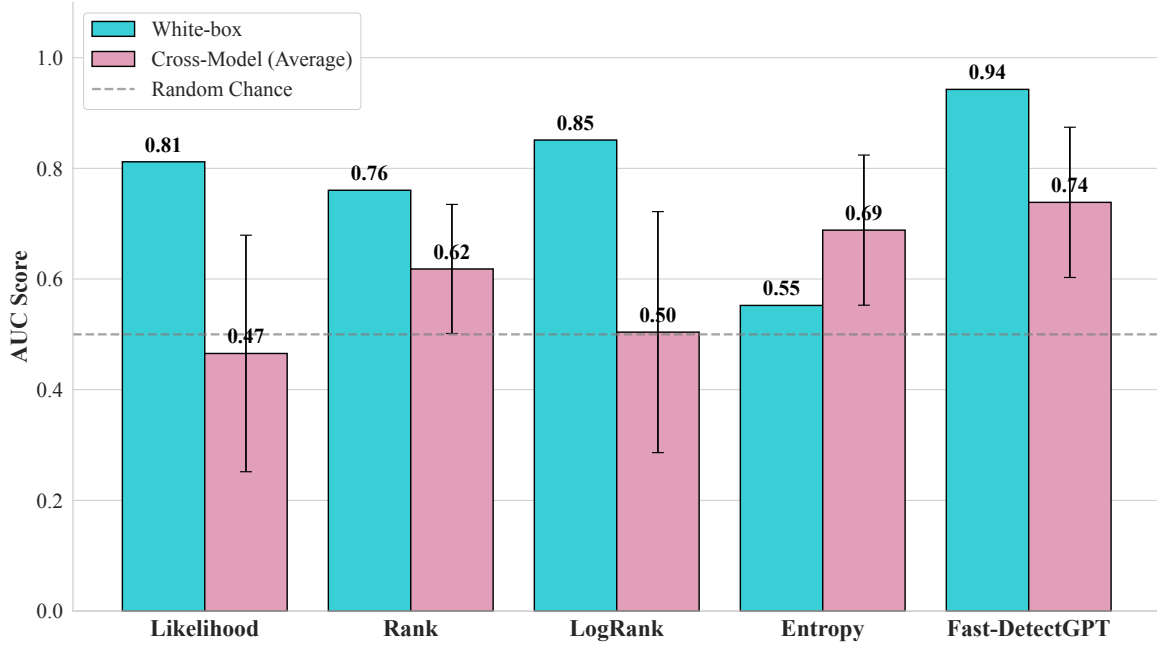


Figure 8. The performance gap across detection methods. Comparison of white-box versus cross-model detection performance across five zero-shot metrics on XSum. The drastic drop in mean AUC and the increase in variance for the cross-model setting demonstrate that fixed surrogates systematically fail to generalize. This visualization corresponds to the quantitative analysis in Table 1.

Table 20. Ablation on the Stage 2 alignment objective. Average AUROC is reported in percentage.

Alignment Objective	EvoBench	MAGE
L_1 regression	87.07	75.56
L_2 regression	86.52	75.18
Cosine similarity	87.43	74.91
KL divergence (ours)	90.85	77.92

D.1. Training Data Construction

Dataset Overview. Our training is based on the MIRAGE benchmark (Fu et al., 2025a), the most comprehensive multi-task machine-generated text detection evaluation framework to date. MIRAGE samples human-written text from ten corpora across five domains: Academic from ArXiv and NeurIPS, News from CNN DailyMail and XSum, Comment from Amazon Review, Email from Enron Emails, and Website from OpenWebText. The benchmark employs 17 mainstream LLMs comprising 13 proprietary and 4 open-source models, and defines three generation tasks: *Generate* for creating new text based on the first 30 tokens of human-written text, *Polish* for refining existing text while preserving details and meaning, and *Rewrite* for paraphrasing without altering semantic content. All texts are cleaned by removing special characters and filtered to 90–220 words to control for length-based detection biases.

Stage 1: Prototype Training Data. For discriminative prototype construction, we synthesize router-aligned training data based on the MIRAGE-DIG subset with disjoint-input generation. For each entry, we treat the original field as ground-truth human text and prompt locally cached language models to produce outputs for the Generate, Rewrite, and Polish tasks. The resulting records follow the same JSON schema as the source data for seamless integration into the existing pipeline. We generate 10,000 samples per task category, yielding 30,000 training instances in total.

Stage 2: Distance Regression Data. For black-box generalization training, we directly utilize the MIRAGE benchmark with detection scores annotated by running each text through our detector pool. Each sample is scored by all combinations of surrogate models and detection criteria, producing a score vector that serves as the regression target for learning text-detector affinity patterns.

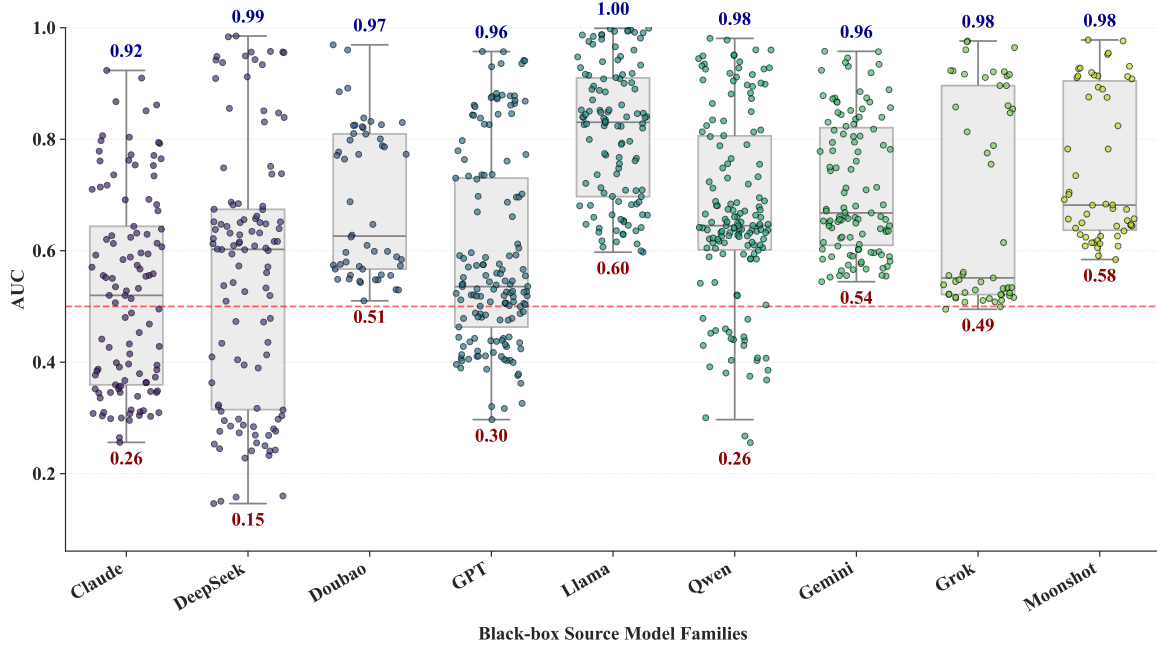


Figure 9. Detailed black-box performance variance. Evaluations on the MIRAGE benchmark (Fu et al., 2025a) using the Fast-DetectGPT criterion. Each data point represents the AUC achieved by a specific open-source surrogate when detecting text from a black-box generator family. The large vertical spread within each column reveals that for any unknown source, there exists an optimal surrogate in our pool achieving high AUC, but a random choice is likely to fail with AUC near the red dashed baseline. This extreme variance motivates the need for adaptive surrogate selection.

D.2. Evaluation Benchmarks

We evaluate on two representative in-the-wild benchmarks that together assess cross-model generalization and temporal robustness.

MAGE Benchmark. MAGE (Li et al., 2024) provides broad coverage across five domains including QA, Comment, News, Academic, and Story with diverse source models, representing established evaluation protocols in machine-generated text detection. Table 21 summarizes the test set organization by source model family. The benchmark comprises 30,265 test pairs spanning eight model families, enabling comprehensive evaluation of cross-model generalization.

EvoBench Benchmark. EvoBench (Yu et al., 2025) specifically targets the evolving nature of LLMs, capturing both updates from version changes by publishers and developments from optimizations by developers such as fine-tuning and pruning. GPT-4o, for example, receives updates approximately every three months. The benchmark covers seven LLM families across 29 evolving versions, with five domain datasets spanning XSum, Writing, PubMed, SocialMedia, and PeerRead across continuation-based, question-answering, and paraphrase generation paradigms. This benchmark directly tests whether routing mechanisms maintain robustness as source models evolve, a critical requirement for real-world deployment. Table 22 presents the test set statistics.

Evaluation Metric. Following standard practice in machine-generated text detection (Mitchell et al., 2023; Bao et al., 2024; Fu et al., 2025a), we adopt the Area Under the Receiver Operating Characteristic Curve as our primary evaluation metric. AUROC measures the probability that a randomly chosen machine-generated text receives a higher detection score than a randomly chosen human-written text, providing a threshold-independent assessment of detector discriminability. This metric ranges from 0.5 for random guessing to 1.0 for perfect separation.

D.3. Surrogate Model Pool

Our white-box surrogate pool comprises ten open-source language models spanning diverse architectures, training corpora, and parameter scales from 1.5B to 20B parameters.

Table 21. MAGE benchmark test set statistics by model family.

Model Family	Test Pairs
OpenAI	8,180
OPT	7,956
FLAN	4,629
LLaMA	3,684
BigScience	2,672
EleutherAI	1,436
GLM	913
Human Paraphrase	795
Total	30,265

Table 22. EvoBench benchmark test set statistics by model family. Datasets indicates the number of model-version and domain combinations.

Model Family	Datasets	Test Pairs
LLaMA-2-7B	74	11,100
GPT4o	35	5,250
Claude	30	4,500
GPT4	28	4,200
LLaMA3	28	4,200
Gemini	15	2,243
Qwen	10	1,500
Total	220	32,993

DeepSeek-7B (DeepSeek-AI, 2023) is a 7B parameter dense transformer optimized for efficient inference. Falcon-7B (Almazrouei et al., 2023) contains 7B parameters and was trained on RefinedWeb with multi-query attention. GPT-oss-20B (OpenAI, 2025) is a 20B parameter open-source GPT variant with standard architecture. GPT-J-6B (Wang & Komatsuzaki, 2021) is an autoregressive transformer with 6B parameters trained on The Pile by EleutherAI. GPT-Neo-2.7B (Black et al., 2021) is a 2.7B parameter EleutherAI model with local attention patterns. GPT-NeoX-20B (Biderman et al., 2023) contains 20B parameters with rotary position embeddings and parallelized attention. GPT2-XL (Radford et al., 2019) is the 1.5B parameter OpenAI baseline trained on WebText. LLaMA3-8B (Dubey et al., 2024) is an 8B parameter Meta instruction-tuned model with grouped-query attention. OPT-2.7B (Zhang et al., 2022) is a 2.7B parameter Meta open pre-trained transformer. Qwen-3B (Bai et al., 2023) is a 3B parameter Alibaba multilingual model.

This diversity ensures that learned prototypes capture a broad spectrum of generation patterns across different model families, scales, and architectural choices.

D.4. Detection Methods

We evaluate six zero-shot detection criteria, each representing a different statistical perspective on distinguishing machine-generated from human-written text.

Likelihood (Solaiman et al., 2019) computes the average log-probability of tokens under the surrogate model, based on the intuition that machine-generated text tends to have higher likelihood under language models. Entropy measures token-level predictive uncertainty, which is typically higher for human text due to its greater unpredictability. Rank (Gehrmann et al., 2019) computes the average vocabulary rank of generated tokens, where machine-generated text tends to use higher-ranked tokens. LogRank (Mitchell et al., 2023) applies logarithmic transformation to token ranks for improved stability. LLR (Su et al., 2023a) computes the log-likelihood ratio between surrogate and reference models to capture relative probability differences. Fast-DetectGPT (Bao et al., 2024) approximates conditional probability curvature without requiring expensive perturbations.

For each criterion, we compare the best fixed-surrogate baseline against our routed variant, which dynamically selects the optimal surrogate per input.

D.5. Router Encoder Architectures

We explore three encoder architectures for the routing network, trading off between computational efficiency and representational capacity.

RoBERTa-base (Liu et al., 2019) is our default choice with 125M parameters in a 12-layer Transformer architecture, offering efficient inference at 4.87ms per sample. RoBERTa-large provides stronger representations with 355M parameters in a 24-layer Transformer. LLM2Vec-LLaMA3-8B (BehnamGhader et al., 2024) adapts the LLaMA3-8B decoder-only LLM for dense embeddings via masked next-token prediction and unsupervised contrastive learning, achieving the best overall performance at the cost of 166.2ms per sample.

D.6. Hyperparameter Configuration

Prototype Settings. We configure 10 learnable prototypes per class, using cosine similarity with temperature $\tau = 10$ and L2-normalized features. Prototypes are initialized via Xavier initialization. We apply inter-class separation with margin 0.5 and weight 10^{-3} , plus prototype norm regularization with weight 10^{-4} .

Stage 1 Training. We train for 8 epochs with batch size 32, learning rate 2×10^{-5} , weight decay 0.01, and 200 warmup steps. The loss consists solely of cross-entropy classification loss.

Stage 2 Training. We train for 8 epochs with batch size 32, reduced learning rate 10^{-5} , weight decay 0.01, and 200 warmup steps. The total loss combines cross-entropy loss with weight 1.0, distance regression loss with weight 1.0, and prototype anchoring loss with weight 1.0 to prevent manifold collapse during transfer.

E. Analysis Experimental Setup

This section provides detailed experimental configurations for the surrogate-source affinity analysis presented in Section 3.

E.1. White-box Cross-Evaluation Protocol

Dataset Construction. Following the protocol established by Fast-DetectGPT (Bao et al., 2024), we use the XSum dataset (Narayan et al., 2018) as our seed corpus. XSum contains 226,711 news articles from BBC with single-sentence summaries. We randomly sample 500 articles from the test split and truncate each to its first 30 tokens as generation prompts. Each of the nine surrogate models then generates continuations of 150–200 tokens using nucleus sampling with $p = 0.96$ and temperature $T = 1.0$, matching the original Fast-DetectGPT configuration. This yields nine parallel corpora of machine-generated text, each containing 500 samples.

Surrogate Model Pool. The white-box analysis employs nine open-source LLMs spanning diverse architectures and scales: GPT-Neo-2.7B (Black et al., 2021), GPT-J-6B (Wang & Komatsuzaki, 2021), GPT-NeoX-20B (Biderman et al., 2023), OPT-2.7B (Zhang et al., 2022), LLaMA3-8B (Dubey et al., 2024), Falcon-7B (Almazrouei et al., 2023), DeepSeek-7B (DeepSeek-AI, 2023), Qwen-3B (Bai et al., 2023), and GPT2-XL (Radford et al., 2019). Parameter counts range from 1.5B to 20B, covering both earlier architectures like GPT-2 and GPT-Neo families as well as more recent models like LLaMA3 and DeepSeek.

Cross-Evaluation Matrix. We construct a 9×9 evaluation matrix where each row represents a source model and each column represents a surrogate model. For each source-surrogate pair, we compute detection scores on the 500 machine-generated samples from the source model using the surrogate model, paired with 500 human-written samples from the original XSum articles. We report AUROC for each cell, yielding 81 evaluation points that reveal the full landscape of cross-model detection performance.

Detection Criteria. The white-box analysis evaluates five zero-shot detection criteria: Likelihood, Entropy, Rank, LogRank, and Fast-DetectGPT. For each criterion, we construct the complete 9×9 affinity matrix. Figure 2 in the main paper visualizes the Fast-DetectGPT matrix; additional matrices for other criteria exhibit similar diagonal-dominant patterns with criterion-specific variations.

E.2. Black-box Evaluation Protocol

Benchmark Selection. For black-box analysis, we utilize the MIRAGE benchmark (Fu et al., 2025a), which provides machine-generated text from major commercial LLMs including GPT-4, GPT-4o, Claude, Gemini, DeepSeek, Grok, LLaMA, Qwen, and GLM. This benchmark covers five domains and three generation tasks, offering comprehensive coverage of real-world black-box scenarios.

Surrogate Sweep. We evaluate each of the nine open-source surrogate models on all black-box source families in MIRAGE. For each source-surrogate combination, we compute AUROC using the Fast-DetectGPT criterion. This yields a performance distribution for each source family, from which we extract the minimum, median, and maximum AUROC across surrogates.

Performance Gap Computation. The performance gap reported in Figure 1 is computed as the relative difference between the best and worst surrogate for each source family: $\text{Gap} = (\text{Max} - \text{Min}) / \text{Max} \times 100\%$. This metric quantifies the potential improvement from optimal surrogate selection compared to an unfortunate random choice.

E.3. Quantitative Summary Statistics

Table 1 in the main paper aggregates the cross-evaluation results into summary statistics. For the white-box setting, we report the average diagonal AUROC as the matched condition and the average off-diagonal AUROC as the cross condition. For the black-box setting, we report the mean AUROC across all source-surrogate pairs, along with the average of per-source maximum and minimum AUROC values. These statistics are computed separately for each detection criterion to reveal criterion-specific sensitivity to surrogate selection.

F. Limitations

We discuss several limitations of the proposed approach.

First, DetectRouter introduces additional computational overhead during inference. The router requires encoding each input text and computing distances to all prototypes before selecting the optimal detector. While lightweight encoders such as RoBERTa-base achieve acceptable latency at 4.87ms per sample, the best-performing LLM2Vec encoder incurs 166.2ms per sample, which may limit applicability in high-throughput scenarios.

Second, the effectiveness of our routing framework depends on the coverage of the surrogate model pool. When encountering text from a source model that is distributionally distant from all surrogates in the pool, the router may fail to identify a well-matched detector. Expanding the surrogate pool can mitigate this issue but increases both training and inference costs.

Third, Stage 2 training requires detection scores from diverse black-box sources as supervision. Collecting such annotations demands running multiple detectors across large-scale datasets, which incurs substantial computational cost during the data preparation phase.

Finally, our current evaluation focuses on English text. The generalization of DetectRouter to multilingual settings remains unexplored and may require language-specific surrogate pools or cross-lingual adaptation strategies.